Supplement S2. Matrix Community Model Projection Example

This example focuses on recreating the analyses published in Lytle *et al.* (2017). In that study, the authors developed population matrices for five guilds of plants in the Colorado Basin, and projected these matrices forward assuming density dependence driven by space limitations affecting recruitment. They asked how changes in climate and hydrology might affect the plant community in this arid region, focusing just on the dominant plant guilds. The specific guilds investigated included: hydroriparian trees (exemplified by cottonwood), xeroriparian shrubs (exemplified by tamarisk), hydroriparian shrubs (exemplified by willow), mesoriparian meadow, and desert shrub (exemplified by sagebrush). Due to some differences in assumptions, our results will be qualitatively similar but not exactly equivalent to the published results. Note that in this example we will often refer to these guilds by their exemplar species.

PART 1. Develop MPMs

First we will clear the memory and load packages `lefko3` and `adapt3`.

```
rm(list=ls(all=TRUE))

library(lefko3)
library(adapt3)
```

Our analysis utilizes a dataset called the `maybell` dataset in Lytle *et al.* (2017), which is posted online at https://doi.org/10.6084/m9.figshare.4652608 at the time of writing. The dataset provides the values of environmental variables collected from 1916 to 1998 for portions of the Colorado Basin, and is used to create the matrices that will be projected using code published at the same site. The dataset is small, and so we replicate it here so that users do not need to find it online.

```
flowdata <- data.frame(year = c(1916:1998))
flowdata$floodmag <- c(11700, 17900, 10500,  7670, 16000, 17700, 10800, 10900,
  7810, 6640, 9090, 11800, 13700, 14400, 7980, 6500, 12100, 11200, 4080, 9870,
  10600, 10000, 12100, 7860, 9170, 11700, 9930, 9280, 9080, 10900, 6850, 12400,
  11300, 9730, 8210, 8870, 13800, 10100, 5480, 7000, 9870, 15700, 12200, 6690,
  8000, 6350, 11500, 6290, 9990, 11800, 6900, 8890, 11400, 8290, 12700, 10300,
  8890, 12100, 15400, 11700, 7450, 3620, 11570, 13600, 11700, 6490, 10700,
  13400, 25100, 13600, 11000, 6140, 10200, 4940, 7020, 8560, 5900, 12800, 5880,
  13300, 15000, 18800, 10700)
flowdata$flooddates <- c(223, 231, 258, 233, 239, 259, 240, 240, 259, 235, 242,
  231, 225, 226, 244, 231, 236, 246, 223, 259, 230, 229, 231, 219, 226, 227,
  239, 246, 237, 225, 211, 222, 234, 262, 237, 242, 249, 258, 235, 228, 237,
  252, 242, 253, 227, 243, 226, 224, 234, 258, 167, 239, 250, 208, 235, 244,
  253, 234, 223, 252, 235, 248, 256, 242, 236, 252, 217, 242, 229, 219, 251,
  230, 231, 225, 256, 235, 240, 234, 231, 260, 231, 247, 234)
flowdata$slopecot <- c(-2.93, -2.33, -1.88, -3.18, -4.62, -4.39, -3.14, -2.92,
  -2.29, -2.03, -3.00, -2.16, -4.26, -2.34, -3.53, -2.15, -1.97, -4.12, -2.36,
  -2.87, -3.36, -1.86, -3.69, -3.28, -3.48, -2.76, -3.67, -2.92, -3.60, -1.81,
  -1.62, -1.89, -3.30, -2.80, -2.62, -3.02, -3.39, -3.34, -1.96, -2.38, -3.64,
  -1.44, -4.70, -2.45, -2.76, -3.28, -1.42, -2.29, -2.36, -1.90, -2.61, -2.51,
  -3.34, -2.44, -3.17, -3.32, -3.15, -1.41, -3.48, -1.05, -2.27, -1.98, -2.28,
  -3.11, -3.05, -2.70, -1.99, -2.82, -3.57, -3.16, -2.92, -1.29, -2.79, -2.44,
  -2.23, -2.75, -1.59, -3.03, -3.16, -0.88, -2.69, -4.33, -2.71)
```

```r
flowdata$slopetam <- c(-1.28, -3.42, -2.40, -0.68, -1.95, -1.71, -1.33, -1.62,
  -1.16, -0.82, -0.68, -1.89, -1.45, -2.21, -0.57, -0.81, -1.94, -1.35, -0.05,
  -1.73, -1.14, -0.97, -1.57, -0.92, -0.72, -1.00, -1.21, -1.93, -1.70, -1.68,
  -0.92, -1.77, -1.12, -1.96, -1.57, -1.74, -1.24, -1.11, -0.72, -0.98, -0.68,
  -3.46, -1.04, -1.49, -1.00, -0.77, -2.31, -0.35, -1.82, -1.78, -0.70, -1.85,
  -1.72, -1.40, -2.30, -2.14, -1.00, -2.09, -1.68, -2.38, -1.31, 0.12, -2.62,
  -2.06, -1.60, -0.94, -2.48, -2.80, -2.41, -1.22, -1.86, -0.44, -1.41, -0.69,
  -1.23, -0.81, -0.52, -2.17, -0.62, -2.63, -1.84, -1.69, -1.97)
flowdata$basedates <- c(295, 315, 306, 279, 307, 318, 293, 310, 294, 291, 292,
  294, 300, 319, 279, 279, 316, 288, 253, 291, 283, 297, 295, 280, 280, 285,
  288, 290, 289, 328, 286, 303, 287, 303, 292, 305, 293, 295, 275, 281, 281,
  331, 283, 286, 284, 277, 300, 273, 293, 314, 273, 301, 296, 299, 305, 303,
  283, 308, 302, 309, 290, 263, 309, 304, 292, 270, 312, 329, 319, 311, 309,
  269, 284, 273, 286, 280, 273, 303, 271, 316, 296, 326, 312)
```

Next we will build the core matrices using the code provided in Lytle *et al.* (2017). These are 83 annual matrices per guild, with five guilds in total. The code below is copied from the originally published code (including remarks) at the time of writing, although we have made a number of changes. First, we have corrected a small number of errors in the published code, after checking with the original authors. Next, we altered the code to produce matrices assuming maximal fecundity, rather than creating matrices with fecundity already adjusted for the aggregate community density. In our analysis, these matrices will be projected later and adjusted at each time step for aggregate density. We have also fit the portions of the code that build matrices into a function called `matrix_creator()` in order to simplify some of the code for users, and have modified some of the formatting.

In the next block, we introduce the first part of the published and edited code. This part gives key values that will be used later to produce the core matrices. We have remarked out unnecessary statements from the original code rather than deleting them, to help users identify differences.

```r
#count <- 83
#replicates <- 1
#outerreps <- 1
total_years <- 83

cseedfirst = 260 # COT: first date of seedset, June 17th or 260
cseedlast = 298 # COT: last date of seedset, 298 or July 25.
tseedfirst = 280 # TAM: first date of seedset, which is July 7th or 280
tseedlast = 336 # TAM: last date of seedset, 336 or 1 Sept.

# FUNCTIONS FOR INCLUDING INSIDE LOOPS ----------------------------------------------------

# Assign either the first date of cottonwood seedset or the first date of flooding
# Cottonwood
csf_func <- function(x) {
    ifelse(cseedfirst > x, cseedfirst, x)
}
cfirstdate <- csf_func(flowdata$flooddates)

# Assign either the last date of cottonwood seedset or the date that baseflow occurs
csl_func <- function(x) {
    ifelse(cseedlast > x, x, cseedlast)
}
clastdate <- csl_func(flowdata$basedates)

# Calculate the number of days in the seedset period
```

```r
cdifference <- clastdate - cfirstdate

# Check to make sure value is positive, assigns 0 otherwise
gzero_func <- function(x) {
    ifelse(x > 0, x, 0)
}

# Length of season for seedset - with negative values removed
cseasonlength <- gzero_func(cdifference)

# Proportion of season available
cproportion <- cseasonlength/(cseedlast - cseedfirst)

# Now repeat all this to get p<tam>
tsf_func <- function(x) {
    ifelse(tseedfirst > x, tseedfirst, x)
}
tfirstdate <- tsf_func(flowdata$flooddates)
tsl_func <- function(x) {
    ifelse(tseedlast > x, x, tseedlast)
}
tlastdate <- tsl_func(flowdata$basedates)
tdifference <- tlastdate - tfirstdate
tseasonlength <- gzero_func(tdifference)
tproportion <- tseasonlength/(tseedlast - tseedfirst)

# FLOOD THRESHOLD FUNCTIONS ----------------------------------------------------------------
# Currently these are the same for cot and tam
# flowdata$floodmag - vector containing peak flood magnitude

# Magnitude of peak flow over which is considered a mortality causing flood event
mortcutoff <- 9888 # This is in CFS, as are the Maybell data points
# Floods above 280 cms 9888 cfs cause mortality, below cause none

# Same for drought (threshold below rather than above)
droughtcutoff <- 7416 # floods above 210 cms / 7416cfs do not cause drought mortality

# Convert peak discharge values into a vector of floods/no floods,
# 1 = an above threshold flood (some scouring), 0 = below threshold (no scouring)
bigflood_func <- function(x) {
    ifelse(x > mortcutoff, 1, 0)
}

# Another similar vector for droughts
# Both of these appear in the functions for vital rates and fecundity, below
drought_func <- function(x) {
    ifelse(x < droughtcutoff, 1, 0)
}

# Vector containing 1's for years that do not have a flood, e.g. normal and drought years
nonflood_func <- function(x) {
    ifelse(x > mortcutoff, 0, 1)
}
```

```r
bigflood <- bigflood_func(flowdata$floodmag)
drought <- drought_func(flowdata$floodmag)
nonflood <- nonflood_func(flowdata$floodmag)

# COT: vector containing the NEGATIVE VALUE OF slopes of all declining limbs of floods
flooddecline <- -flowdata$slopecot
# COT: drawdown survival, a lognormal FUNCTION.
# Positive values of h indicate receding water
cdds_func <- function(x) {
  .944 * exp(-.5 * ((log(x/1.279))/.987)^2)
}
decline <- cdds_func(flooddecline) # flood decline is the h value from paper

# TAM: vector containing NEGATIVE VALUE OF slopes of all declining limbs of floods
tflooddecline <- -flowdata$slopetam
# TAM: drawdown survival, a lognormal FUNCTION
tdds_func <- function(x) {
  0.917 * exp(-.5 * ((x - 1.8)/3.4)^2)
}
tdecline <- tdds_func(tflooddecline)

# Cottonwood core rates
# Stage specific densities
denC1 <- 350
denC2 <- 10
denC3 <- 1
denC4 <- .91
denC5 <- .6
denC6 <- .12

# "Self thinning" rates, or equivalency rules, for stage transitions
bC1 <- denC2/denC1
bC2 <- denC3/denC2
bC3 <- denC4/denC3
bC4 <- denC5/denC4
bC5 <- denC6/denC5

# Baseline maturation probabiliity, aC6 (adult senescence rate)
aC1 <- 1
aC2 <- 1
aC3 <- 1
aC4 <- 1
aC5 <- .167
aC6 <- .03

# Initial area in m2. Here, it is calculated based ONLY on cottonwood.
areaC1 <- 11816
areaC2 <- 11816
areaC3 <- 11144
areaC4 <- 11144
areaC5 <- 13819
areaC6 <- 59605
```

```r
K <- (areaC1 + areaC2 + areaC3 + areaC4 + areaC5 + areaC6) * denC1

# Tamarisk core rates
# # Stage specific densities, number per m2
denT1 <- 400
denT2 <- 29
denT3 <- 4.5
denT4 <- 1.4
denT5 <- 1.3
denT6 <- 1.3

# "Self thinning" rates, or equivalency rules, for stage transitions
bT1 <- denT2/denT1
bT2 <- denT3/denT2
bT3 <- denT4/denT3
bT4 <- denT5/denT4
bT5 <- denT6/denT5

# Baseline maturation probability, aT6 (adult senescence rate)
aT1 <- 1
aT2 <- 1
aT3 <- .25
aT4 <- .11
aT5 <- .07
aT6 <- .05

# Willow core rates
# Stage specific densities, number per m2
# THESE NUMBERS ARE ARBITRARY, but reflect a final adult size/spacing of 1 m2 per plant
denW1 <- 350
denW2 <- 35
denW3 <- 1
denW4 <- 1
denW5 <- 1
denW6 <- 1

# "Self thinning" rates, or equivalency rules, for stage transitions
bW1 <- denW2/denW1
bW2 <- denW3/denW2
bW3 <- denW4/denW3
bW4 <- denW5/denW4
bW5 <- denW6/denW5

# Baseline maturation probability, aW6 is adult senescence rate
# Only aW6 is different from cottonwood (faster)
aW1 <- 1
aW2 <- 1
aW3 <- 1
aW4 <- 1
aW5 <- .167
aW6 <- .01

# Sagebrush core rates
```

```r
# Stage specific densities, number per m2
# THESE NUMBERS ARE ARBITRARY, but reflect an adult size/spacing of 1 m2
denS1 <- 350
denS2 <- 35
denS3 <- 1
denS4 <- 1
denS5 <- 1
denS6 <- 1

# "Self thinning" rates, or equivalency rules, for stage transitions
bS1 <- denS2/denS1
bS2 <- denS3/denS2
bS3 <- denS4/denS3
bS4 <- denS5/denS4
bS5 <- denS6/denS5

# Baseline maturation probabiliity, aS6 is adult senescence rate.
# SAME AS COTTONWOOD
aS1 <- 1
aS2 <- 1
aS3 <- 1
aS4 <- 1
aS5 <- .167
aS6 <- .03

# Meadow core rates
# Stage specific densities, number per m2
# THESE NUMBERS ARE ARBITRARY, but reflect an adult size/spacing of 1 m2
denM1 <- 350
denM2 <- 35
denM3 <- 1
denM4 <- 1
denM5 <- 1
denM6 <- 1

# "Self thinning" rates, or equivalency rules, for stage transitions
bM1 <- denM2/denM1
bM2 <- denM3/denM2
bM3 <- denM4/denM3
bM4 <- denM5/denM4
bM5 <- denM6/denM5

# Baseline maturation probability, aM6 is adult senescence rate. SAME AS COTTONWOOD
aM1 <- 1
aM2 <- 1
aM3 <- 1
aM4 <- 1
aM5 <- .167
aM6 <- .03
```

Next, we introduce a function that will use the values above and the `maybell` dataset to create the core matrices to use in projection. We have developed this function with sensitivity analysis in mind, and will explain this later in the example. Here, we note that the following argument definitions:

`flowdata` refers to the core dataset used to create the matrices, which is the `maybell` dataset by default. `total_years` refers to the total number of annual matrices to create. `SC6_offset` is a numeric additive offset to the flood mortality for the oldest class cottonwood (hydroriparian tree) individuals. `ST6_offset` is a numeric additive offset to the flood mortality for the oldest class tamarisk (xeroriparian shrub) individuals. `SW6_offset` is a numeric additive offset to the flood mortality for the oldest class willow (hydroriparian shrub) individuals. `SS6_offset` is a numeric additive offset to the flood mortality for the oldest class sagebrush (desert shrub) individuals. `SM6_offset` is a numeric additive offset to the flood mortality for the oldest class meadow (mesoriparian) individuals. `DC6_offset` is a numeric additive offset to the drought mortality for the oldest class cottonwood (hydroriparian tree) individuals. `DT6_offset` is a numeric additive offset to the drought mortality for the oldest class tamarisk (xeroriparian shrub) individuals. `DW6_offset` is a numeric additive offset to the drought mortality for the oldest class willow (hydroriparian shrub) individuals. `DS6_offset` is a numeric additive offset to the drought mortality for the oldest class sagebrush (desert shrub) individuals. `DM6_offset` is a numeric additive offset to the drought mortality for the oldest class meadow (mesoriparian) individuals.

```r
matrix_creator <- function(flowdata = flowdata, total_years = 83, SC6_offset = 0,
  ST6_offset = 0, SW6_offset = 0, SS6_offset = 0, SM6_offset = 0, DC6_offset = 0,
  DT6_offset = 0, DW6_offset = 0, DS6_offset = 0, DM6_offset = 0) {
  # ITERATION PARAMETERS ---------------------------------------------------------------------
  # Setting up arrays/vectors to fill with data from loops

  # Inner loop details -----------------------------------------------------------------------
  # 'count' - number of years to project simulations (inner loop)

  # Output of no. ind. for each age class for each year projected into the future
  # An array with 6 columns (each age class) and however many rows there are years projected
  # Coutput <- array(0, dim = c(count, 6)) # Cottonwood
  # Toutput <- array(0, dim = c(count, 6)) # Tamarisk
  # Woutput <- array(0, dim = c(count, 6)) # Willow
  # Soutput <- array(0, dim = c(count, 6)) # Sagebrush
  # Moutput <- array(0, dim = c(count, 6)) # Meadow

  # Total cottonwood pop. size as % of K
  # This is the total space occupied by this species in cottonwood seedling units
  # Cspaceoutput <- numeric(length = count) # Cottonwood
  # Tspaceoutput <- numeric(length = count) # Tamarisk
  # Wspaceoutput <- numeric(length = count) # Willow
  # Sspaceoutput <- numeric(length = count) # Sagebrush
  # Mspaceoutput <- numeric(length = count) # Meadow

  # Flood and drought settings for each year projected into the future (i.e. 0 or 1)
  # floodoutput <- numeric(length = count) # flood
  # droughtoutput <- numeric(length = count) # drought
  # nonfloodoutput <- numeric(length = count) # nonflood
  # normaloutput <- numeric(length = count) # normal

  # Total pop. size as % of K WITHOUT SEEDLINGS for each year projected into future
  # contains %K of each guild except for seedlings for each year of projection
  # Cnonseedling <- numeric(length = count)
  # Tnonseedling <- numeric(length = count)
  # Wnonseedling <- numeric(length = count)
  # Snonseedling <- numeric(length = count)
  # Mnonseedling <- numeric(length = count)
```

```
# No. ind. at stages as per Merritt and Poff 2010
# Vector of cot age class 5 for each year projected
#DomC <- numeric(length = count) # these will record the NUMBER of individuals in stg 5,
# which are 5-10 year olds
#DomT <- numeric(length = count) # same, for stages 4, although this is 7-15 year olds

# Mid loop details ---------------------------------------------------------------------
# 'replicates' - number of replicate projections to run (mid loop)

# Mean values for each rep. run over the period specified from burnin to end of projection
# Mean density of each guild WITHOUT seedlings included - each replicate run
# Crep <- numeric(length = replicates)
# Trep <- numeric(length = replicates)
# Wrep <- numeric(length = replicates)
# Srep <- numeric(length = replicates)
# Mrep <- numeric(length = replicates)

# Mean density of each guild WITH seedlings included
# Crep_all <- numeric(length = replicates)
# Trep_all <- numeric(length = replicates)
# Wrep_all <- numeric(length = replicates)
# Srep_all <- numeric(length = replicates)
# Mrep_all <- numeric(length = replicates)

# Mean of DomC for each of the replicate runs
#DomCrep <- numeric(length = replicates) # to record output of DomC from flow scenarios
#DomTrep <- numeric(length = replicates)

# Outer loop details -------------------------------------------------------------------
# 'outerreps' - number of iterations for outer loop that alters drought/flood frequency
# Results of flow mod scenarios. Not useful unless simulating changes to flow regime
# This is the mean of each flow mod. setting for the full burnin->end of projection period

# No seedlings for each of the flow mod settings
# Cgraph <- numeric(length = outerreps)
# Tgraph <- numeric(length = outerreps)
# Mgraph <- numeric(length = outerreps)
# Wgraph <- numeric(length = outerreps)
# Sgraph <- numeric(length = outerreps)

# All incl. seedlings for each of the flow mod settings
# Cgraph_all <- numeric(length = outerreps)
# Tgraph_all <- numeric(length = outerreps)
# Mgraph_all <- numeric(length = outerreps)
# Wgraph_all <- numeric(length = outerreps)
# Sgraph_all <- numeric(length = outerreps)

# Mean of DomC for each of the flow mod settings
#DomCrep_graph <- numeric(length = outerreps)
#DomTrep_graph <- numeric(length = outerreps)

# Proportion of flow years in each flow mod setting
#droughtgraph <- numeric(length = outerreps) # droughts
```

```r
#floodgraph <- numeric(length = outerreps) # floods
#nonfloodgraph <- numeric(length = outerreps) # nonfloods
#normalgraph <- numeric(length = outerreps) # normal years

# Setting flow scenario change to none to begin with
# The outer loop iterates these one step at a time until it reaches 'outerreps'
#droughtchanged = floodchanged = 0

# RECRUITMENT AS FUNCTION OF FLOOD DECLINE RATE ----------------------------------------
# Checks to see if at least one adult is present
# adult_func <- function(x) {
#     ifelse(x > .99999, 1, 0)
# }

# Keeps FC6 from dividing by zero by substituting an arbitrary nonzero number that will get
# multiplied by zero later anyway during matrix multiplication
# nonind <- function(x) {
#     ifelse(x == 0, 666, x)
# }

# checkpos makes sure that the K-occupied term is positive, assigns 0 if not
# checkpos <- function(x) {
#     ifelse(x < 0, 0, x)
# }

# Quasi extinction threshold of 1, keeps pop from asymptoting infinitely to zero
# quasi <- function(x) {
#     ifelse(x < 1, 0, x)
# }

# Quasi rescue function that keeps species from disappearing,
# e.g. sagebrush that can encroach from uplands
# quasiten <- function(x) {
#     ifelse(x < 10, 10, x)
# }

# VITAL RATES - COTTONWOOD ------------------------------------------------------------

# Flood mortality in a flood year
SC1 <- .97
SC2 <- .33
SC3 <- .224
SC4 <- .19
SC5 <- .073
SC6 <- .02 + SC6_offset

# Drought mortality in a drought year
DC1 <- .49
DC2 <- .16
DC3 <- .083
DC4 <- .05
DC5 <- .05
DC6 <- .05 + DC6_offset
```

```r
# VITAL RATES - TAMARISK ---------------------------------------------------------------------

# Flood mortality in a flood year
ST1 <- .9
ST2 <- .55
ST3 <- .25
ST4 <- .05
ST5 <- .01
ST6 <- .01 + ST6_offset

# Drought mortality in a drought year
DT1 <- .5
DT2 <- .15
DT3 <- .05
DT4 <- .025
DT5 <- .025
DT6 <- .025 + DT6_offset

# Initial area in m2. Not included in initial K calculation
areaT1 <- 1000
areaT2 <- 0
areaT3 <- 0
areaT4 <- 0
areaT5 <- 0
areaT6 <- 0

# VITAL RATES - willow ------------------------------------------------------------------------

# Flood mortality in a flood year. HALF THAT OF COTTONWOOD
SW1 <- .49
SW2 <- .17
SW3 <- .11
SW4 <- .10
SW5 <- .04
SW6 <- .01 + SW6_offset

# Drought mortality in a drought year
# Basically, SURVIVORSHIP rate is half that of cottonwood: convert mortality to
# survivorship, divide by two, convert back to mortality
DW1 <- .75
DW2 <- .58
DW3 <- .54
DW4 <- .53
DW5 <- .51
DW6 <- .51 + DW6_offset

# Initial area in m2. Not included in initial K calculation
areaW1 <- 100
areaW2 <- 100
areaW3 <- 100
areaW4 <- 100
areaW5 <- 100
areaW6 <- 100
```

```r
# VITAL RATES - sagebrush ------------------------------------------------------------------

# Flood mortality in a flood year
# Taken by taking survivorship of cot (1-S), dividing by 2, and coverting back to mort.
# So, flood SURVIVORSHIP is 1/2 that of cottonwood
SS1 <- .99
SS2 <- .67
SS3 <- .61
SS4 <- .60
SS5 <- .54
SS6 <- .51 + SS6_offset

# Drought MORTALITY in a drought year. ONE HALF COTTONWOOD RATES
DS1 <- .24
DS2 <- .08
DS3 <- .042
DS4 <- .025
DS5 <- .005
DS6 <- .005 + DS6_offset

# Initial area in m2. Not included in initial K calculation
areaS1 <- 100
areaS2 <- 100
areaS3 <- 100
areaS4 <- 100
areaS5 <- 100
areaS6 <- 100

# VITAL RATES - Meadow ---------------------------------------------------------------------

# Flood mortality in a flood year
# Taken by taking survivorship of cot (1-S), dividing by 2, and coverting back to mort.
# FOR STAGES 1-3 ONLY
# So, flood SURVIVORSHIP is 1/2 that of cottonwood for those stages, reflecting instablity
# of meadow habitats in highly flood-prone situations.
SM1 <- .99
SM2 <- .67
SM3 <- .61
SM4 <- .19
SM5 <- .073
SM6 <- .02 + SM6_offset

# Drought MORTALITY in a drought year
# ONE HALF COTTONWOOD RATES in stages 4-6 only, reflecting greater drought tolerance in
# established meadows and less groundwater dependence
DM1 <- .49
DM2 <- .16
DM3 <- .083
DM4 <- .025
DM5 <- .005
DM6 <- .005 + DM6_offset

# Initial area in m2. Not included in initial K calculation
```

```r
areaM1 <- 100
areaM2 <- 100
areaM3 <- 100
areaM4 <- 100
areaM5 <- 100
areaM6 <- 100

# Maybell area in m2, as established from initial cottonwood occupancy
# K is the total area available for cot OR tam expressed in cottonwood seedlings per m2
NC <- c(areaC1 * denC1,
        areaC2 * denC2,
        areaC3 * denC3,
        areaC4 * denC4,
        areaC5 * denC5,
        areaC6 * denC6)
# NC gives the total number of individuals for each age class.
# Initially here, this is found by multiplying the number of m2 occupied by a given class
# by the the density per m2

NT <- c(areaT1 * denT1,
        areaT2 * denT2,
        areaT3 * denT3,
        areaT4 * denT4,
        areaT5 * denT5,
        areaT6 * denT6)

NW <- c(areaW1 * denW1,
        areaW2 * denW2,
        areaW3 * denW3,
        areaW4 * denW4,
        areaW5 * denW5,
        areaW6 * denW6)

NS <- c(areaS1 * denS1,
        areaS2 * denS2,
        areaS3 * denS3,
        areaS4 * denS4,
        areaS5 * denS5,
        areaS6 * denS6)

NM <- c(areaM1 * denM1,
        areaM2 * denM2,
        areaM3 * denM3,
        areaM4 * denM4,
        areaM5 * denM5,
        areaM6 * denM6)

# HT: Cottonwood. Includes anything with 'C' or 'Cot'. e.g. 'Cgraph', 'Crep', 'DomC'
# XS: Tamarisk. Includes anything with 'T' or 'Tam'. e.g. 'Tgraph', 'Trep', 'DomT'
# HS: Willow. Includes anything with 'W'. e.g. 'Wgraph', 'Wrep'
# MM: Meadow. Includes anything with 'M'. e.g. 'Mgraph', 'Mrep'
# DS: Sagebrush. Includes anything with 'S'. e.g. 'Sgraph', 'Srep'
```

```r
HTlist <- vector(mode = "list", length = total_years)
XSlist <- vector(mode = "list", length = total_years)
HSlist <- vector(mode = "list", length = total_years)
MMlist <- vector(mode = "list", length = total_years)
DSlist <- vector(mode = "list", length = total_years)

for(i in 1:total_years) {

  y <- i # sample(nrow(flowdata), 1)
  # y is a random number within the length of the flow data to randomly select a year from
  # the 'bigflood' and 'drought' vector
  # in this case anything between 1 and 83 in the maybell data

  # VITAL RATE DEFINITIONS: cottonwood -----------------------------------------------------
  # G is prob. of transition to next stage
  # P is prob. of remaining in that stage
  GC1 <- aC1 * bC1 * (1 - bigflood[y] * SC1) * (1 - drought[y] * DC1)
  GC2 <- aC2 * bC2 * (1 - bigflood[y] * SC2) * (1 - drought[y] * DC2)
  GC3 <- aC3 * bC3 * (1 - bigflood[y] * SC3) * (1 - drought[y] * DC3)
  GC4 <- aC4 * bC4 * (1 - bigflood[y] * SC4) * (1 - drought[y] * DC4)
  GC5 <- aC5 * bC5 * (1 - bigflood[y] * SC5) * (1 - drought[y] * DC5)
  PC5 <- (1 - aC5) * (1 - bigflood[y] * SC5) * (1 - drought[y] * DC5)
  PC6 <- (1 - aC6) * (1 - bigflood[y] * SC6) * (1 - drought[y] * DC6)

  # VITAL RATE DEFINITIONS: tamarisk -------------------------------------------------------
  GT1 <- aT1 * bT1 * (1 - bigflood[y] * ST1) * (1 - drought[y] * DT1)
  GT2 <- aT2 * bT2 * (1 - bigflood[y] * ST2) * (1 - drought[y] * DT2)
  GT3 <- aT3 * bT3 * (1 - bigflood[y] * ST3) * (1 - drought[y] * DT3)
  GT4 <- aT4 * bT4 * (1 - bigflood[y] * ST4) * (1 - drought[y] * DT4)
  GT5 <- aT5 * bT5 * (1 - bigflood[y] * ST5) * (1 - drought[y] * DT5)
  PT3 <- (1 - aT3) * (1 - bigflood[y] * ST3) * (1 - drought[y] * DT3)
  PT4 <- (1 - aT4) * (1 - bigflood[y] * ST4) * (1 - drought[y] * DT4)
  PT5 <- (1 - aT5) * (1 - bigflood[y] * ST5) * (1 - drought[y] * DT5)
  PT6 <- (1 - aT6) * (1 - bigflood[y] * ST6) * (1 - drought[y] * DT6)

  # VITAL RATE DEFINITIONS: willow ---------------------------------------------------------
  GW1 <- aW1 * bW1 * (1 - bigflood[y] * SW1) * (1 - drought[y] * DW1)
  GW2 <- aW2 * bW2 * (1 - bigflood[y] * SW2) * (1 - drought[y] * DW2)
  GW3 <- aW3 * bW3 * (1 - bigflood[y] * SW3) * (1 - drought[y] * DW3)
  GW4 <- aW4 * bW4 * (1 - bigflood[y] * SW4) * (1 - drought[y] * DW4)
  GW5 <- aW5 * bW5 * (1 - bigflood[y] * SW5) * (1 - drought[y] * DW5)
  PW5 <- (1 - aW5) * (1 - bigflood[y] * SW5) * (1 - drought[y] * DW5)
  PW6 <- (1 - aW6) * (1 - bigflood[y] * SW6) * (1 - drought[y] * DW6)

  # VITAL RATE DEFINITIONS: sagebrush ------------------------------------------------------
  GS1 <- aS1 * bS1 * (1 - bigflood[y] * SS1) * (1 - drought[y] * DS1)
  GS2 <- aS2 * bS2 * (1 - bigflood[y] * SS2) * (1 - drought[y] * DS2)
  GS3 <- aS3 * bS3 * (1 - bigflood[y] * SS3) * (1 - drought[y] * DS3)
  GS4 <- aS4 * bS4 * (1 - bigflood[y] * SS4) * (1 - drought[y] * DS4)
  GS5 <- aS5 * bS5 * (1 - bigflood[y] * SS5) * (1 - drought[y] * DS5)
  PS5 <- (1 - aS5) * (1 - bigflood[y] * SS5) * (1 - drought[y] * DS5)
  PS6 <- (1 - aS6) * (1 - bigflood[y] * SS6) * (1 - drought[y] * DS6)
```

```r
# VITAL RATE DEFINITIONS: meadow ------------------------------------------------------------
GM1 <- aM1 * bM1 * (1 - bigflood[y] * SM1) * (1 - drought[y] * DM1)
GM2 <- aM2 * bM2 * (1 - bigflood[y] * SM2) * (1 - drought[y] * DM2)
GM3 <- aM3 * bM3 * (1 - bigflood[y] * SM3) * (1 - drought[y] * DM3)
GM4 <- aM4 * bM4 * (1 - bigflood[y] * SM4) * (1 - drought[y] * DM4)
GM5 <- aM5 * bM5 * (1 - bigflood[y] * SM5) * (1 - drought[y] * DM5)
PM5 <- (1 - aM5) * (1 - bigflood[y] * SM5) * (1 - drought[y] * DM5)
PM6 <- (1 - aM6) * (1 - bigflood[y] * SM6) * (1 - drought[y] * DM6)

# FECUNDITY ---------------------------------------------------------------------------------
# Assumes that if any breeding tam or cot is present, they will seed all recently-scoured
# substrates, although success is scaled by the lognormal hydrograph drawdown functions.
# If cot and tam seedlings behave entirely independently as modeled here, this means that
# under the right conditions there can be overseeding such that if (K-occupied) is the
# total amount of scoured habitat remaining, then up to 2*(K-occupied) could be colonized
# by seedlings, half tam and half cot. One consequence of this is if an extended drought
# is followed by a good flood and then many growth years, populations could exceed K for
# many years, and when pop > K, recruitment will not occur (although flood-related
# mortality will keep lowering population sizes).

# Post flood space occupied -----------------------------------------------------------------
# postfloodC gives the amount of space occupied by cottonwood after the flood (AND
# ACTUALLY AFTER DROUGHT OR NORMAL YEARS AS WELL!!!) IN COTTONWOOD SEEDLING UNITS
# postfloodC <-
#     NC[1] * GC1/bC1 +
#     NC[2] * GC2/(bC2 * bC1) +
#     NC[3] * GC3/(bC3 * bC2 * bC1) +
#     NC[4] * GC4/(bC4 * bC3 * bC2 * bC1) +
#     NC[5] * PC5/(bC4 * bC3 * bC2 * bC1) +
#     NC[5] * GC5/(bC5 * bC4 * bC3 * bC2 * bC1) +
#     NC[6] * PC6/(bC5 * bC4 * bC3 * bC2 * bC1)
#
# # postfloodT gives the amount of space occupied by tamarisk after the flood
# # IN TAMARISK SEEDLING UNITS
# postfloodT  <-
#     NT[1] * GT1/bT1 +
#     NT[2] * GT2/(bT2 * bT1) +
#     NT[3] * GT3/(bT3 * bT2 * bT1) +
#     NT[3] * PT3/(bT2 * bT1) +
#     NT[4] * GT4/(bT4 * bT3 * bT2 * bT1) +
#     NT[4] * PT4/(bT3 * bT2 * bT1) +
#     NT[5] * GT5/(bT5 * bT4 * bT3 * bT2 * bT1) +
#     NT[5] * PT5/(bT4 * bT3 * bT2 * bT1) +
#     NT[6] * PT6/(bT5 * bT4 * bT3 * bT2 * bT1)
#
# # postfloodW gives the amount of space occupied by willow after the flood
# # IN WILLOW SEEDLING UNITS
# postfloodW <-
#     NW[1] * GW1/bW1 +
#     NW[2] * GW2/(bW2 * bW1) +
#     NW[3] * GW3/(bW3 * bW2 * bW1) +
#     NW[4] * GW4/(bW4 * bW3 * bW2 * bW1) +
#     NW[5] * PW5/(bW4 * bW3 * bW2 * bW1) +
```

```
#     NW[5] * GW5/(bW5 * bW4 * bW3 * bW2 * bW1) +
#     NW[6] * PW6/(bW5 * bW4 * bW3 * bW2 * bW1)
#
# # postfloodS gives the amount of space occupied by SAGEBRUSH after the flood
# # IN SAGEBRUSH SEEDLING UNITS
# postfloodS <-
#     NS[1] * GS1/bS1 +
#     NS[2] * GS2/(bS2 * bS1) +
#     NS[3] * GS3/(bS3 * bS2 * bS1) +
#     NS[4] * GS4/(bS4 * bS3 * bS2 * bS1) +
#     NS[5] * PS5/(bS4 * bS3 * bS2 * bS1) +
#     NS[5] * GS5/(bS5 * bS4 * bS3 * bS2 * bS1) +
#     NS[6] * PS6/(bS5 * bS4 * bS3 * bS2 * bS1)
#
# # postfloodM gives the amount of space occupied by meadow after the flood
# # IN MEADOW SEEDLING UNITS
# postfloodM <-
#     NM[1] * GM1/bM1 +
#     NM[2] * GM2/(bM2 * bM1) +
#     NM[3] * GM3/(bM3 * bM2 * bM1) +
#     NM[4] * GM4/(bM4 * bM3 * bM2 * bM1) +
#     NM[5] * PM5/(bM4 * bM3 * bM2 * bM1) +
#     NM[5] * GM5/(bM5 * bM4 * bM3 * bM2 * bM1) +
#     NM[6] * PM6/(bM5 * bM4 * bM3 * bM2 * bM1)


# POTENTIAL COTTONWOOD FECUNDITY ----------------------------------------------------------
# FC6 <- checkpos((adult_func(NC[6])) * # checks to see if at least 1 adult is present
#                (1/nonind(NC[6])) *
#                bigflood[y] *
#                cproportion[y] *
#                decline[y] *
#                (K - (postfloodC +
#                      postfloodT * (denC1/denT1) +
#                      postfloodW * (denC1/denW1) +
#                      postfloodS * (denC1/denS1) +
#                      postfloodM * (denC1/denM1))))

FC6max <- (#checkpos((adult_func(NC[6])) *
               #(1/nonind(NC[6])) *
               bigflood[y] *
               cproportion[y] *
               decline[y] *
               (K))

# '(1/nonind(NC[6]))' keeps FC6 from dividing by zero by substituting an arbitrary non-0
# number that will be multiplied by 0 later anyway during matrix multiplication

# This gives POTENTIAL MAX fecundity based on amount of bare substrate available AFTER
# that year's flood.
# Reproduction is conditional on:
# 1. at least one reproductive cottonwood being present
# 2. a big flood occurring
```

```
# 3. flood during seedset window.
# It is independent of # of repro adults, but it is scaled by the rate of flooddecline AND
# BY THE PROPORTION OF SEEDSET DAYS.
# Number of new seedlings is determined by the total amount of bare substrates;
# i.e. whatever is not occupied by surviving cottonwood OR OTHER SPECIES.
# The denC1/denT1 term converts tam to cot seedling units and likewise for other 3 guilds.
# The 1/nonind[NC[6]] term will cancel out with NC[6] during matrix projection.
# i.e. number of seedlings is independent of number of mature trees.

# POTENTIAL TAMARISK FECUNDITY ------------------------------------------------------------
 # FT <- checkpos((adult_func(NT[3] + NT[4] + NT[5] + NT[6])) *
 #               bigflood[y] *
 #               tproportion[y] *
 #               tdecline[y] *
 #               (denT1/denC1) *
 #               (K - (postfloodC +
 #                     postfloodT * (denC1/denT1) +
 #                     postfloodW * (denC1/denW1) +
 #                     postfloodS * (denC1/denS1) +
 #                     postfloodM * (denC1/denM1))))

FTmax <- (#checkpos((adult_func(NT[3] + NT[4] + NT[5] + NT[6])) *
               bigflood[y] *
               tproportion[y] *
               tdecline[y] *
               (denT1/denC1) *
               (K))

# This gives fecundities of TAMARISK seedlings based on amount of bare substrates
# available AFTER that year's flood.
# Note that units are cot seedlings in the (K-occupied) term, but are then converted back
# to tam seedlings.
# Repro is conditional on
# 1. at least one repro tamarisk being present,
# 2. a big flood occurring
# 3. during seedset.
# It is independent of # of repro adults, but it is scaled by the rate of flooddecline and
# by prop of seedset days.
# Note that the independence of FT from breeding pop size is achieved differently here
# than for cot, it is "forced" during matrix iteration, below.

# WILLOW FECUNDITY ------------------------------------------------------------------------
# FW <- checkpos((adult_func(NW[3] + NW[4] + NW[5] + NW[6])) *
#                bigflood[y] *
#                (denW1/denC1) *
#                (K - (postfloodC +
#                      postfloodT * (denC1/denT1) +
#                      postfloodW * (denC1/denW1) +
#                      postfloodS * (denC1/denS1) +
#                      postfloodM * (denC1/denM1))))

FWmax <- (#checkpos((adult_func(NW[3] + NW[4] + NW[5] + NW[6])) *
               bigflood[y] *
```

```
                (denW1/denC1) *
                (K))

# checks to make sure at least one stage 3 to 6 age individual is present, and that a
# flood occurs. No dependence on drawdown or seedset timing.

# SAGEBRUSH FECUNDITY ------------------------------------------------------------------
# FS <- checkpos((adult_func(NS[2] + NS[3] + NS[4] + NS[5] + NS[6])) *
#                nonflood[y] *
#                (denS1/denC1) *
#                (K - (postfloodC +
#                      postfloodT * (denC1/denT1) +
#                      postfloodW * (denC1/denW1) +
#                      postfloodS * (denC1/denS1) +
#                      postfloodM * (denC1/denM1))))

FSmax <- (#checkpos((adult_func(NS[2] + NS[3] + NS[4] + NS[5] + NS[6])) *
                nonflood[y] *
                (denS1/denC1) *
                (K))

# Sagebrush fecundity in # of sagebrush seedlings, to be added using "placeholder" to NS1
# during iteration.
# Here, sagebrush can colonize any empty portion of K, but only during NONFLOOD years.

# MEADOW FECUNDITY ---------------------------------------------------------------------
# FM <- checkpos((adult_func(NM[2] + NM[3] + NM[4] + NM[5] + NM[6])) *
#                bigflood[y] *
#                (denM1/denC1) *
#                (K - (postfloodC +
#                      postfloodT * (denC1/denT1) +
#                      postfloodW * (denC1/denW1) +
#                      postfloodS * (denC1/denS1) +
#                      postfloodM * (denC1/denM1))))

FMmax <- (#checkpos((adult_func(NM[2] + NM[3] + NM[4] + NM[5] + NM[6])) *
                bigflood[y] *
                (denM1/denC1) *
                (K))

# Meadow fecundity in # of seedlings, to be added using "placeholder" to NM1 during iter.
# Here, meadow can colonize any empty portion of K.

# K ------------------------------------------------------------------------------------
# Cottonwood
# gives total cottonwood population size as a percentage of K;
# this is the total space occupied by this species in cottonwood seedling units
# KC <- 100 * (NC[1] +
#              NC[2]/(bC1) +
#              NC[3]/(bC2 * bC1) +
#              NC[4]/(bC3 * bC2 * bC1) +
#              NC[5]/(bC4 * bC3 * bC2 * bC1) +
#              NC[6]/(bC5 * bC4 * bC3 * bC2 * bC1))/K
```

```
#
# # same as above, but without seedlings
# CnonseedK <- 100 * (NC[2]/(bC1) +
#                     NC[3]/(bC2 * bC1) +
#                     NC[4]/(bC3 * bC2 * bC1) +
#                     NC[5]/(bC4 * bC3 * bC2 * bC1) +
#                     NC[6]/(bC5 * bC4 * bC3 * bC2 * bC1))/K
# # Tamarisk
# KT <- 100 * (denC1/denT1) * (NT[1] +
#                              NT[2]/(bT1) +
#                              NT[3]/(bT2 * bT1) +
#                              NT[4]/(bT3 * bT2 * bT1) +
#                              NT[5]/(bT4 * bT3 * bT2 * bT1) +
#                              NT[6]/(bT5 * bT4 * bT3 * bT2 * bT1))/K
#
# TnonseedK <- 100 * (denC1/denT1) * (NT[2]/(bT1) +
#                                     NT[3]/(bT2 * bT1) +
#                                     NT[4]/(bT3 * bT2 * bT1) +
#                                     NT[5]/(bT4 * bT3 * bT2 * bT1) +
#                                     NT[6]/(bT5 * bT4 * bT3 * bT2 * bT1))/K
# # Willow
# KW <- 100 * (NW[1] +
#              NW[2]/(bW1) +
#              NW[3]/(bW2 * bW1) +
#              NW[4]/(bW3 * bW2 * bW1) +
#              NW[5]/(bW4 * bW3 * bW2 * bW1) +
#              NW[6]/(bW5 * bW4 * bW3 * bW2 * bW1))/K
#
# WnonseedK  <- 100 * (NW[2]/(bW1) +
#                      NW[3]/(bW2 * bW1) +
#                      NW[4]/(bW3 * bW2 * bW1) +
#                      NW[5]/(bW4 * bW3 * bW2 * bW1) +
#                      NW[6]/(bW5 * bW4 * bW3 * bW2 * bW1))/K
# # Sagebrush
# KS  <- 100 * (NS[1] +
#               NS[2]/(bS1) +
#               NS[3]/(bS2 * bS1) +
#               NS[4]/(bS3 * bS2 * bS1) +
#               NS[5]/(bS4 * bS3 * bS2 * bS1) +
#               NS[6]/(bS5 * bS4 * bS3 * bS2 * bS1))/K
#
# SnonseedK <- 100 * (NS[2]/(bS1) +
#                     NS[3]/(bS2 * bS1) +
#                     NS[4]/(bS3 * bS2 * bS1) +
#                     NS[5]/(bS4 * bS3 * bS2 * bS1) +
#                     NS[6]/(bS5 * bS4 * bS3 * bS2 * bS1))/K
# # Meadow
# KM <- 100 * (NM[1] +
#              NM[2]/(bM1) +
#              NM[3]/(bM2 * bM1) +
#              NM[4]/(bM3 * bM2 * bM1) +
#              NM[5]/(bM4 * bM3 * bM2 * bM1) +
#              NM[6]/(bM5 * bM4 * bM3 * bM2 * bM1))/K
```

```r
#
# MnonseedK <- 100 * (NM[2]/(bM1) +
#                     NM[3]/(bM2 * bM1) +
#                     NM[4]/(bM3 * bM2 * bM1) +
#                     NM[5]/(bM4 * bM3 * bM2 * bM1) +
#                     NM[6]/(bM5 * bM4 * bM3 * bM2 * bM1))/K

# TRANSITION MATRICES ------------------------------------------------------------------

# TRANSITION MATRIX FOR cottonwood -----------------------------------------------------
AC1 <- c(0, 0, 0, 0, 0, FC6max)
AC2 <- c(GC1, 0, 0, 0, 0, 0)
AC3 <- c(0, GC2, 0, 0, 0, 0)
AC4 <- c(0, 0, GC3, 0, 0, 0)
AC5 <- c(0, 0, 0, GC4, PC5, 0)
AC6 <- c(0, 0, 0, 0, GC5, PC6)
# Matrix
AC <- rbind(AC1, AC2, AC3, AC4, AC5, AC6)

# TRANSITION MATRIX FOR tamarisk -------------------------------------------------------
# Note: fecundity is not included here, since it is assigned directly during iteration
AT1 <- c(0, 0, FTmax, FTmax, FTmax, FTmax)
AT2 <- c(GT1, 0, 0, 0, 0, 0)
AT3 <- c(0, GT2, PT3, 0, 0, 0)
AT4 <- c(0, 0, GT3, PT4, 0, 0)
AT5 <- c(0, 0, 0, GT4, PT5, 0)
AT6 <- c(0, 0, 0, 0, GT5, PT6)
# Matrix
AT <- rbind(AT1, AT2, AT3, AT4, AT5, AT6)

# TRANSITION MATRIX FOR dynamic riverbank specialist, willow ------------------------------
# Similar stage structure to cot, except that reproduction can occur in all but 1st yr
AW1 <- c(0, 0, FWmax, FWmax, FWmax, FWmax)
AW2 <- c(GW1, 0, 0, 0, 0, 0)
AW3 <- c(0, GW2, 0, 0, 0, 0)
AW4 <- c(0, 0, GW3, 0, 0, 0)
AW5 <- c(0, 0, 0, GW4, PW5, 0)
AW6 <- c(0, 0, 0, 0, GW5, PW6)
# Matrix
AW <- rbind(AW1, AW2, AW3, AW4, AW5, AW6)

# TRANSITION MATRIX FOR sagebrush ------------------------------------------------------
# Arid shrubland indicator big sagebrush.
# Similar stage structure to cottonwood, fecundity is assigned directly as with tamarisk
# during iteration.
AS1 <- c(0, 0, FSmax, FSmax, FSmax, FSmax)
AS2 <- c(GS1, 0, 0, 0, 0, 0)
AS3 <- c(0, GS2, 0, 0, 0, 0)
AS4 <- c(0, 0, GS3, 0, 0, 0)
AS5 <- c(0, 0, 0, GS4, PS5, 0)
AS6 <- c(0, 0, 0, 0, GS5, PS6)
# Matrix
AS <- rbind(AS1, AS2, AS3, AS4, AS5, AS6)
```

```r
# TRANSITION MATRIX FOR 5th species, xeric meadow ----------------------------------------
# Includes grasses such as wheatgrass.
# Similar stage structure to cottonwood, fecundity is assigned directly as with tamarisk
# during iteration.
AM1 <- c(0, 0, FMmax, FMmax, FMmax, FMmax)
AM2 <- c(GM1, 0, 0, 0, 0, 0)
AM3 <- c(0, GM2, 0, 0, 0, 0)
AM4 <- c(0, 0, GM3, 0, 0, 0)
AM5 <- c(0, 0, 0, GM4, PM5, 0)
AM6 <- c(0, 0, 0, 0, GM5, PM6)
# Matrix
AM <- rbind(AM1, AM2, AM3, AM4, AM5, AM6)

# COMPILING OUTPUTS ------------------------------------------------------------------------
# Cottonwood
#Coutput[i,1:6] <- log(NC + 1) # array of no. ind. of each age class for each yr
# projected. NC = total no. ind. for each age class
#DomC[i] <- NC[5] # vector of cot age class 5 for each year projected
#Cnonseedling[i] <- CnonseedK # total cottonwood pop. size as % of K WITHOUT SEEDLINGS for
# each projected year
#Cspaceoutput[i] <- KC # total cottonwood pop. size as % of K; this is the total space
# occupied by this species in cottonwood seedling units

# Tamarisk - same as cottonwood
#Toutput[i,1:6] <- log(NT + 1)
#DomT[i] <- NT[4]
#Tnonseedling[i] <- TnonseedK
#Tspaceoutput[i] <- KT

# Willow
#Woutput[i,1:6] <- log(NW + 1)
#Wnonseedling[i] <- WnonseedK
#Wspaceoutput[i] <- KW

# Sagebrush
#Soutput[i,1:6] <- log(NS + 1)
#Snonseedling[i] <- SnonseedK
#Sspaceoutput[i] <- KS

# Meadow
#Moutput[i,1:6] <- log(NM + 1)
#Mnonseedling[i] <- MnonseedK
#Mspaceoutput[i] <- KM

# Records flood settings of each particular projected year (0 for nonflood, 1 for flood)
#floodoutput[i] <- bigflood[y]

# Same for drought
#droughtoutput[i] <- drought[y]

# Same for nonflood
#nonfloodoutput[i] <- nonflood[y]
```

```
    # Same for normal
    #normaloutput[i] <- ifelse(bigflood[y] == 0 & drought[y] == 0 & nonflood[y] == 1, 1, 0)

    # Fecundity of all but cottonwood to put into matrix projection.
    # Cottonwood is already in matrix
    #Tplaceholder <- FTmax
    #Splaceholder <- FSmax
    #Wplaceholder <- FWmax
    #Mplaceholder <- FMmax

    # MATRIX MULTIPLICATION ------------------------------------------------------------------
    # Cottonwood
    #NC <- AC %*% NC # AC is transition matrix, NC = total no. ind. for each age class
    #NC <- quasi(NC) # quasi extinction threshold of 1: below 1 go to 0

    # Tamarisk
    # Note the use of 'placeholders' for fecundity in the following guilds
    #NT <- AT %*% NT
    #NT[1] <- Tplaceholder
    #NT <- quasi(NT)

    # Willow
    #NW <- AW %*% NW
    #NW[1] <- Wplaceholder
    #NW <- quasi(NW)

    # Sagebrush
    #NS <- AS %*% NS
    #NS[1] <- Splaceholder
    #NS <- quasiten(NS)

    # Meadow
    #NM <- AM %*% NM
    #NM[1] <- Mplaceholder
    #NM <- quasi(NM)

    HTlist[[y]] <- AC
    XSlist[[y]] <- AT
    HSlist[[y]] <- AW
    MMlist[[y]] <- AM
    DSlist[[y]] <- AS

  } # End of inner loop

  out_list <- list(cottonwood = HTlist, tamarisk = XSlist, willow = HSlist,
    sagebrush = DSlist, meadow = MMlist)
}
```

Finally, we will run the function above and create the core matrices, which will be stored in a nested list.

```
riparian_mats <- matrix_creator(flowdata, total_years)
```

Now we have one main list with five elements, one for each guild. Each of these is in turn a list of 83 matrices. To use these lists in `adapt3`, we will need to create `lefkoMat` objects to hold them. These

objects are structured lists used in the population matrix projection package `lefko3`, and they include lists of matrices belonging to one species or population (or in this case, guild), along with metadata describing the matrices such as population names, years covered by each matrix, a description of the life history that defines all stages included in the matrices, and, if necessary, a matrix row / column index. In the code below, the life history model is described with function `sf_create()`, and the `lefkoMat` object is created with function `create_lM()`, both from package `lefko3`.

```
tree_size <- c(1:6) # The actual size values do not matter, but non-overlapping values are required
tree_stage <- c("Sdl", "2yr", "3yr", "4yr", "sub", "adult")
tree_rep <- c(0, 0, 0, 0, 0, 1)
tree_mat <- c(0, 0, 0, 0, 0, 1)
tree_imm <- c(1, 1, 1, 1, 1, 0)

tree_frame <- sf_create(sizes = tree_size, stagenames = tree_stage,
  repstatus = tree_rep, matstatus = tree_mat, immstatus = tree_imm)

cottonwood_lM <- create_lM(riparian_mats$cottonwood, tree_frame, yearorder = c(1:83))


mead_size <- c(1:6) # The actual size values do not matter, but non-overlapping values are required
mead_stage <- c("Sdl", "2yr", "3yr", "7yr", "16yr", "over30yr")
mead_rep <- c(0, 0, 1, 1, 1, 1)
mead_mat <- c(0, 0, 1, 1, 1, 1)
mead_imm <- c(1, 1, 0, 0, 0, 0)

mead_frame <- sf_create(sizes = mead_size, stagenames = mead_stage,
  repstatus = mead_rep, matstatus = mead_mat, immstatus = mead_imm)

tamarisk_lM <- create_lM(riparian_mats$tamarisk, mead_frame, yearorder = c(1:83))
sagebrush_lM <- create_lM(riparian_mats$sagebrush, mead_frame, yearorder = c(1:83))
meadow_lM <- create_lM(riparian_mats$meadow, mead_frame, yearorder = c(1:83))


will_size <- c(1:6) # The actual size values do not matter, but non-overlapping values are required
will_stage <- c("Sdl", "2yr", "3yr", "7yr", "16yr", "over30yr")
will_rep <- c(0, 1, 1, 1, 1, 1)
will_mat <- c(0, 1, 1, 1, 1, 1)
will_imm <- c(1, 0, 0, 0, 0, 0)

will_frame <- sf_create(sizes = will_size, stagenames = will_stage,
  repstatus = will_rep, matstatus = will_mat, immstatus = will_imm)

willow_lM <- create_lM(riparian_mats$willow, will_frame, yearorder = c(1:83))
```

PART 2. Weight Individuals

We now have the core matrices to project, which include maximal fecundity unadjusted for aggregate density. However, the matrix community model (MCM) that we would like to project still needs further information. Particularly, we need to specify stage equivalence within and across MPMs, the start vectors for projection, and the nature of density dependence. The former information will allow the projection to develop a common, aggregate community density in which the numbers of individuals in the stage of a particular guild are properly weighted against those of other stages within the same and other guilds. The latter will adjust density-dependent matrix elements by the aggregate density.

First, we will build data frames giving the stage equivalence. In a typical study, one data frame would

be built per species. However, Lytle *et al.* (2017) utilizes equivalence weights that occasionally shift due to the impact of changing environmental parameters. So, we will build five lists, one for each guild, with each holding 83 such equivalence data frames, one for each matrix in corresponding order.

```
cottonwood_eq_list <- vector(mode = "list", length = total_years)
tamarisk_eq_list <- vector(mode = "list", length = total_years)
willow_eq_list <- vector(mode = "list", length = total_years)
sagebrush_eq_list <- vector(mode = "list", length = total_years)
meadow_eq_list <- vector(mode = "list", length = total_years)

for (i in c(1:total_years)) {
  # Cottonwood
  GC1 <- cottonwood_lM$A[[i]][2, 1]
  GC2 <- cottonwood_lM$A[[i]][3, 2]
  GC3 <- cottonwood_lM$A[[i]][4, 3]
  GC4 <- cottonwood_lM$A[[i]][5, 4]
  GC5 <- cottonwood_lM$A[[i]][6, 5]
  PC5 <- cottonwood_lM$A[[i]][5, 5]
  PC6 <- cottonwood_lM$A[[i]][6, 6]

  stage_vec_C_eq <- c(1, 2, 3, 4, 5, 6)
  value_vec_C_eq <- c(GC1/bC1, GC2/(bC2 * bC1), GC3/(bC3 * bC2 * bC1),
    GC4/(bC4 * bC3 * bC2 * bC1), (PC5/(bC4 * bC3 * bC2 * bC1) +
      GC5/(bC5 * bC4 * bC3 * bC2 * bC1)), PC6/(bC5 * bC4 * bC3 * bC2 * bC1))

  cottonwood_eq <- stage_weight(cottonwood_lM, stage2 = stage_vec_C_eq,
    value = value_vec_C_eq) #postfloodC
  cottonwood_eq_list[[i]] <- cottonwood_eq

  # Tamarisk
  GT1 <- tamarisk_lM$A[[i]][2, 1]
  GT2 <- tamarisk_lM$A[[i]][3, 2]
  GT3 <- tamarisk_lM$A[[i]][4, 3]
  GT4 <- tamarisk_lM$A[[i]][5, 4]
  GT5 <- tamarisk_lM$A[[i]][6, 5]
  PT3 <- tamarisk_lM$A[[i]][3, 3]
  PT4 <- tamarisk_lM$A[[i]][4, 4]
  PT5 <- tamarisk_lM$A[[i]][5, 5]
  PT6 <- tamarisk_lM$A[[i]][6, 6]

  stage_vec_T_eq <- c(1, 2, 3, 4, 5, 6)
  value_vec_T_eq <- c(((denC1 / denT1) * GT1/bT1),
    ((denC1 / denT1) * GT2/(bT2 * bT1)),
    (((denC1 / denT1) * GT3/(bT3 * bT2 * bT1)) +
      ((denC1 / denT1) * PT3/(bT2 * bT1))),
    (((denC1 / denT1) * GT4/(bT4 * bT3 * bT2 * bT1)) +
      ((denC1 / denT1) * PT4/(bT3 * bT2 * bT1))),
    (((denC1 / denT1) * GT5/(bT5 * bT4 * bT3 * bT2 * bT1)) +
      ((denC1 / denT1) * PT5/(bT4 * bT3 * bT2 * bT1))),
    ((denC1 / denT1) * PT6/(bT5 * bT4 * bT3 * bT2 * bT1)))

  tamarisk_eq <- stage_weight(tamarisk_lM, stage2 = stage_vec_T_eq,
    value = value_vec_T_eq) #postfloodT
  tamarisk_eq_list[[i]] <- tamarisk_eq
```

```r
# Willow
GW1 <- willow_lM$A[[i]][2, 1]
GW2 <- willow_lM$A[[i]][3, 2]
GW3 <- willow_lM$A[[i]][4, 3]
GW4 <- willow_lM$A[[i]][5, 4]
GW5 <- willow_lM$A[[i]][6, 5]
PW5 <- willow_lM$A[[i]][5, 5]
PW6 <- willow_lM$A[[i]][6, 6]

stage_vec_W_eq <- c(1, 2, 3, 4, 5, 6)
value_vec_W_eq <- c(((denC1 / denW1) * GW1/bW1),
  ((denC1 / denW1) * GW2/(bW2 * bW1)),
  ((denC1 / denW1) * GW3/(bW3 * bW2 * bW1)),
  ((denC1 / denW1) * GW4/(bW4 * bW3 * bW2 * bW1)),
  (((denC1 / denW1) * PW5/(bW4 * bW3 * bW2 * bW1)) +
    ((denC1 / denW1) * GW5/(bW5 * bW4 * bW3 * bW2 * bW1))),
  ((denC1 / denW1) * PW6/(bW5 * bW4 * bW3 * bW2 * bW1)))

willow_eq <- stage_weight(willow_lM, stage2 = stage_vec_W_eq,
  value = value_vec_W_eq) #postfloodW
willow_eq_list[[i]] <- willow_eq

# Sagebrush
GS1 <- sagebrush_lM$A[[i]][2, 1]
GS2 <- sagebrush_lM$A[[i]][3, 2]
GS3 <- sagebrush_lM$A[[i]][4, 3]
GS4 <- sagebrush_lM$A[[i]][5, 4]
GS5 <- sagebrush_lM$A[[i]][6, 5]
PS5 <- sagebrush_lM$A[[i]][5, 5]
PS6 <- sagebrush_lM$A[[i]][6, 6]

stage_vec_S_eq <- c(1, 2, 3, 4, 5, 6)
value_vec_S_eq <- c(((denC1 / denS1) * GS1/bS1),
  ((denC1 / denS1) * GS2/(bS2 * bS1)),
  ((denC1 / denS1) * GS3/(bS3 * bS2 * bS1)),
  ((denC1 / denS1) * GS4/(bS4 * bS3 * bS2 * bS1)),
  (((denC1 / denS1) * PS5/(bS4 * bS3 * bS2 * bS1)) +
    ((denC1 / denS1) * GS5/(bS5 * bS4 * bS3 * bS2 * bS1))),
  ((denC1 / denS1) * PS6/(bS5 * bS4 * bS3 * bS2 * bS1)))

sagebrush_eq <- stage_weight(sagebrush_lM, stage2 = stage_vec_S_eq,
  value = value_vec_S_eq) #postfloodS
sagebrush_eq_list[[i]] <- sagebrush_eq

# Meadow
GM1 <- meadow_lM$A[[i]][2, 1]
GM2 <- meadow_lM$A[[i]][3, 2]
GM3 <- meadow_lM$A[[i]][4, 3]
GM4 <- meadow_lM$A[[i]][5, 4]
GM5 <- meadow_lM$A[[i]][6, 5]
PM5 <- meadow_lM$A[[i]][5, 5]
PM6 <- meadow_lM$A[[i]][6, 6]
```

```r
  stage_vec_M_eq <- c(1, 2, 3, 4, 5, 6)
  value_vec_M_eq <- c(((denC1 / denM1) * GM1/bM1),
    ((denC1 / denM1) * GM2/(bM2 * bM1)),
    ((denC1 / denM1) * GM3/(bM3 * bM2 * bM1)),
    ((denC1 / denM1) * GM4/(bM4 * bM3 * bM2 * bM1)),
    (((denC1 / denM1) * PM5/(bM4 * bM3 * bM2 * bM1)) +
      ((denC1 / denM1) * GM5/(bM5 * bM4 * bM3 * bM2 * bM1))),
    ((denC1 / denM1) * PM6/(bM5 * bM4 * bM3 * bM2 * bM1)))

  meadow_eq <- stage_weight(meadow_lM, stage2 = stage_vec_M_eq,
    value = value_vec_M_eq) #postfloodM
  meadow_eq_list[[i]] <- meadow_eq
}
```

Let's see an example of a stage equivalence data frame. This is the first cottonwood equivalence frame, and refers to year 1916.

```r
cottonwood_eq_list[[1]]
```

```
##   stage2 stage_id_2 stage1 stage_id_1 age2 row_num     value
## 1    Sdl          1     NA         NA   NA       1    0.0300
## 2    2yr          2     NA         NA   NA       2   23.4500
## 3    3yr          3     NA         NA   NA       3  271.6000
## 4    4yr          4     NA         NA   NA       4  311.5385
## 5    sub          5     NA         NA   NA       5  540.7500
## 6  adult          6     NA         NA   NA       6 2772.5833
```

The numbers in the `value` column give the equivalence of a single individual in terms of cottonwood seedling units as they occur spatially in the landscape. These values sometimes differ from year to year, as shown here from the cottonwood data, but 3 years later.

```r
cottonwood_eq_list[[4]]
```

```
##   stage2 stage_id_2 stage1 stage_id_1 age2 row_num     value
## 1    Sdl          1     NA         NA   NA       1    1.0000
## 2    2yr          2     NA         NA   NA       2   35.0000
## 3    3yr          3     NA         NA   NA       3  350.0000
## 4    4yr          4     NA         NA   NA       4  384.6154
## 5    sub          5     NA         NA   NA       5  583.3333
## 6  adult          6     NA         NA   NA       6 2829.1667
```

To keep the density of the different guilds comparable, all stages of all guilds are weighted relative to cottonwood seedlings, as below for tamarisk.

```r
tamarisk_eq_list[[1]]
```

```
##     stage2 stage_id_2 stage1 stage_id_1 age2 row_num      value
## 1      Sdl          1     NA         NA   NA       1   0.087500
## 2      2yr          2     NA         NA   NA       2   5.431034
## 3      3yr          3     NA         NA   NA       3  58.333333
## 4      7yr          4     NA         NA   NA       4 237.500000
## 5     16yr          5     NA         NA   NA       5 266.538462
## 6 over30yr          6     NA         NA   NA       6 253.211538
```

PART 3. Set Start Vectorts

Now we will input the start vectors for each of the five populations, as provided in the published code. We can do so with `lefko3`'s function `start_input`, as follows.

```
NC_start <- start_input(cottonwood_lM, stage2 = c("Sdl", "2yr", "3yr", "4yr", "sub", "adult"),
  value = c(4135600.00, 118160.00, 11144.00, 10141.04, 8291.40, 7152.60))
NT_start <- start_input(tamarisk_lM, stage2 = c("Sdl", "2yr", "3yr", "7yr", "16yr", "over30yr"),
  value = c(4e+05, 0e+00, 0e+00, 0e+00, 0e+00, 0e+00))
NW_start <- start_input(willow_lM, stage2 = c("Sdl", "2yr", "3yr", "7yr", "16yr", "over30yr"),
  value = c(35000, 3500, 100, 100, 100, 100))
NS_start <- start_input(sagebrush_lM, stage2 = c("Sdl", "2yr", "3yr", "7yr", "16yr", "over30yr"),
  value = c(35000, 3500, 100, 100, 100, 100))
NM_start <- start_input(meadow_lM, stage2 = c("Sdl", "2yr", "3yr", "7yr", "16yr", "over30yr"),
  value = c(35000, 3500, 100, 100, 100, 100))
```

Let's take a peek at one of these data frames.

```
NC_start
```

```
##   stage2 stage_id_2 stage1 stage_id_1 age2 row_num      value
## 1    Sdl          1     NA         NA   NA       1 4135600.00
## 2    2yr          2     NA         NA   NA       2  118160.00
## 3    3yr          3     NA         NA   NA       3   11144.00
## 4    4yr          4     NA         NA   NA       4   10141.04
## 5    sub          5     NA         NA   NA       5    8291.40
## 6  adult          6     NA         NA   NA       6    7152.60
```

Here we see that the cottonwood population is starting with 4135600 seedlings, 118160 2yr saplings, 11144 3yr saplings, 10141.04 4yr saplings, 8291.4 subadults, and 7152.6 adults.

PART 4. Set Density Dependence

Now let's build the density dependence data frames, which will also be inserted into lists. In this study, the fecundity of all guilds is density dependent, and so must be adjusted in each matrix used in each time step. Under the "recruitment box" model, fecundity only occurs in years in which environmental conditions are right (Mahoney & Rood 1998). These conditions are defined by the authors as being years in which a major flood occurs, at least one tree is present, and seed release occurs during a time period with sufficient soil moisture (as detailed by a hydrograph). The authors define maximal fecundity in terms of the number of new individuals that may be recruited assuming a maximal amount of bare soil surface immediately after a flood.

The style of density dependence is referred to here as the *additive limit* function, which overwrites fecundity after matrix multiplication occurs and replaces it with a stage equivalence-adjusted version of the following function:

$$\phi_{yx} = \beta(\alpha - \sum_j^s \sum_{i \neq y}^m n_{ij,t+1}), \phi_{yx} \in [\gamma, (\alpha - \sum_j^s \sum_{i \neq y}^m n_{ij,t+1})]$$

Here, ϕ_{yx} is the matrix transition element describing the rate of transition from stage x in time t to stage y in time $t+1$, α is the maximum value of fecundity (equivalent to K in equation 1 of Lytle *et al.* (2017)), β is a user-supplied real number ranging from 0 to 1 (equivalent to $p_j(h)g_j(h)$ in equation 1 of Lytle *et al.* (2017)), γ is a user-supplied minimum value, and n is the number of individuals in time $t+1$ of all individuals in all m stages other than the destinaton stage in the current species or guild, and of all s species or guilds (equivalent to $K_{ij}(t+1)$ in equation 1 of Lytle *et al.* (2017)). Because Lytle *et al.* (2017) make fecundity density dependent, the destination stage in all cases is the seedling stage.

```
cottonwood_dens_list <- vector(mode = "list", length = total_years)
tamarisk_dens_list <- vector(mode = "list", length = total_years)
willow_dens_list <- vector(mode = "list", length = total_years)
sagebrush_dens_list <- vector(mode = "list", length = total_years)
```

```r
meadow_dens_list <- vector(mode = "list", length = total_years)

K_used_C <- K_used_T <- K_used_W <- K_used_S <- K_used_M <- 0.

for (i in c(1:total_years)) {
  # Cottonwood
  FC6 <- cottonwood_lM$A[[i]][1, 6]

  if (FC6 > 0.) {
    K_used_C <- K
  } else K_used_C <- 0.

  cottonwood_dens <- density_input(cottonwood_lM, stage3 = "Sdl",
    stage2= "adult", style = 5, alpha = K_used_C,
    beta = (bigflood[i] * cproportion[i] * decline[i]), type = 2)
  cottonwood_dens_list[[i]] <- cottonwood_dens

  # Tamarisk
  FT <- tamarisk_lM$A[[i]][1, 6]

  if (FT > 0.) {
    K_used_T <- K
  } else K_used_T <- 0.

  tamarisk_dens <- density_input(tamarisk_lM, stage3 = "Sdl",
    stage2= "rep", style = 5, alpha = K_used_T,
    beta = (bigflood[i] * tproportion[i] * tdecline[i] * (denT1/denC1)),
    type = 2)
  tamarisk_dens_list[[i]] <- tamarisk_dens

  # Willow
  FW <- willow_lM$A[[i]][1, 6]

  if (FW > 0.) {
    K_used_W <- K
  } else K_used_W <- 0.

  willow_dens <- density_input(willow_lM, stage3 = "Sdl",
    stage2= "rep", style = 5, alpha = K_used_W,
    beta = (bigflood[i] * (denW1/denC1)), type = 2)
  willow_dens_list[[i]] <- willow_dens

  # Sagebrush
  FS <- sagebrush_lM$A[[i]][1, 6]

  if (FS > 0.) {
    K_used_S <- K
  } else K_used_S <- 0.

  sagebrush_dens <- density_input(sagebrush_lM, style = c(5, 6, 6, 6, 6, 6),
    stage3 = c("Sdl", "2yr", "3yr", "7yr", "16yr", "over30yr"),
    stage2= c("rep", "2yr", "3yr", "7yr", "16yr", "over30yr"),
    alpha = K_used_S, gamma = c(10),
```

```
    beta = c((nonflood[i] * (denS1/denC1)), 10, 10, 10, 10, 10),
    type = c(2, 1, 1, 1, 1, 1))
  sagebrush_dens_list[[i]] <- sagebrush_dens

  # Meadow
  FM <- meadow_lM$A[[i]][1, 6]

  if (FM > 0.) {
    K_used_M <- K
  } else K_used_M <- 0.

  meadow_dens <- density_input(meadow_lM, stage3 = "Sdl",
    stage2= "rep", style = 5, alpha = K_used_M,
    beta = (bigflood[i] * (denM1/denC1)), type = 2)
  meadow_dens_list[[i]] <- meadow_dens
}
```

Let's take a look at one of these data frames.

```
cottonwood_dens_list[[1]]
```

```
##   stage3 stage2 stage1 age2 style    alpha      beta gamma time_delay type
## 1    Sdl  adult   <NA>   NA     5 41770400 0.6110757     0          1    2
##   type_t12
## 1        1
```

In our code, we broke down the originally published code to produce the relevant parameters, with a separate density frame per matrix per guild. Typically, this means that *alpha* is the maximal fecundity value in each matrix, and *beta* is a product of some of the original environmental parameters modified by seedling stage equivalents relating all fecundity in cottonwood seedling units. The code above translates this into real numbers usable in the projection.

PART 5. Package MPM Info into Lists

Now we will package all of our inputs into lists. It is important that the lists are in the same MPM order (i.e., the cottonwood (hydroriparian tree) objects are always first, the tamarisk (xeroriparian shrub) is always second, etc.). Here, we package the MPMs into a list called `lytle_mpms`, the equivalence data frames into a list called `lytle_alleq`, the start vectors into a list called `lytle_starts`, and the density dependence information into a list called `lytle_alldens`.

```
lytle_mpms <- list(cottonwood_lM, tamarisk_lM, willow_lM, sagebrush_lM,
  meadow_lM)
lytle_alleq <- list(cottonwood_eq_list, tamarisk_eq_list, willow_eq_list,
  sagebrush_eq_list, meadow_eq_list)
lytle_starts <- list(NC_start, NT_start, NW_start, NS_start, NM_start)
lytle_alldens <- list(cottonwood_dens_list, tamarisk_dens_list,
  willow_dens_list, sagebrush_dens_list, meadow_dens_list)
```

PART 6. Run Projections

Finally, we will run some projections. Our first projection will be a test projection. This will be a short (100 year), density-dependent projection without temporal stochasticity, and we will use it only to explore how `adapt3` projections work. Note that we include all of the lists that we made as inputs into function `project3()`. We finish off the block below with a `summary()` function call that describes the results.

```r
lytle_alleq_alldens_proj <- project3(mpms = lytle_mpms, times = 100,
  equivalence = lytle_alleq, density = lytle_alldens, starts = lytle_starts)

summary(lytle_alleq_alldens_proj, finalN_mean = TRUE, finalN_used = 10)
```

```
##
## The input adaptProj object covers 5 populations, 101 projected steps and 1 projected replicate.
## The number of replicates with population size above the threshold size of 1 is as in
## the following matrix, with populations given by row and milepost times given by column:

## $milepost_sums
##      1 25 50 75 101
## [1,] 1  1  1  1   1
## [2,] 1  1  1  1   1
## [3,] 1  1  1  1   1
## [4,] 1  1  1  1   1
## [5,] 1  1  1  1   1
##
## $extinction_times
## [1] NA
##
## $final_N
##       1       2       3       4       5
## 2326985 1709441 3470355 1138195 3362592
```

The summary output shows that none of the populations were predicted to go extinct in the 100 time steps of the projection. It also shows the mean population size over the final 10 times for each guild, which are all rather high.

Perhaps we would like to see a plot of what actually happened? The structure of the `adaptProj` object created by function `project3()` above allows us to easily extract such data. First, the aggregate community density at each time is given in object `agg_density`, as below.

```r
lytle_alleq_alldens_proj$agg_density
```

```
##          [,1]     [,2]     [,3]     [,4]     [,5]     [,6]     [,7]     [,8]
## [1,] 33800064 35439631 36256418 36623677 53154418 39317274 38001511 37872142
##          [,9]    [,10]    [,11]    [,12]    [,13]    [,14]    [,15]    [,16]
## [1,] 37887771 48879006 34858459 41156113 36516983 36850245 37285674 52437510
##         [,17]    [,18]    [,19]    [,20]    [,21]    [,22]    [,23]    [,24]
## [1,] 35254604 36341152 36596077 34633399 41234952 34188475 35643045 36231478
##         [,25]    [,26]    [,27]    [,28]    [,29]    [,30]    [,31]    [,32]
## [1,] 51341584 50805705 40666971 39297708 42357654 41767506 39126177 33404727
##         [,33]    [,34]    [,35]    [,36]    [,37]    [,38]    [,39]    [,40]
## [1,] 35105501 35910981 50949049 50418870 49887861 44079880 42085296 33765824
##         [,41]    [,42]    [,43]    [,44]    [,45]    [,46]    [,47]    [,48]
## [1,] 35877580 41236011 33197064 33869455 34375436 41264879 37305643 32594244
##         [,49]    [,50]    [,51]    [,52]    [,53]    [,54]    [,55]    [,56]
## [1,] 38430233 32997773 34674889 35367021 41294598 35040325 55250324 38924429
##         [,57]    [,58]    [,59]    [,60]    [,61]    [,62]    [,63]    [,64]
## [1,] 37623626 46306549 38517326 38162992 38212358 49219454 34952789 36162860
##         [,65]    [,66]    [,67]    [,68]    [,69]    [,70]    [,71]    [,72]
## [1,] 36806390 37010987 34066389 34648450 35974663 36530160 36775107 37152514
##         [,73]    [,74]    [,75]    [,76]    [,77]    [,78]    [,79]    [,80]
## [1,] 32766347 33357748 33689441 36073801 41358323 38543109 32345008 38069670
##         [,81]    [,82]    [,83]    [,84]    [,85]    [,86]    [,87]    [,88]
```

```
## [1,] 33194845 34232846 35695719 36044745 36368836 36946214 37196403 51891318
##          [,89]    [,90]    [,91]    [,92]    [,93]    [,94]    [,95]    [,96]
## [1,] 39814033 38595601 38485055 38492473 47838906 32550331 41311228 35021962
##          [,97]    [,98]    [,99]   [,100]   [,101]
## [1,] 35944538 36818854 54901824 33981196 35690746
```

If we had more than one replicate, then these would be given in further rows in the matrix above. Object `N_out` gives each population size in matrix format, with one matrix per replicate. Let's use these data to produce our plot, using the `plot.adaptProj()` function included in package `adapt3`.

```
plot(lytle_alleq_alldens_proj, bty = "n")
```

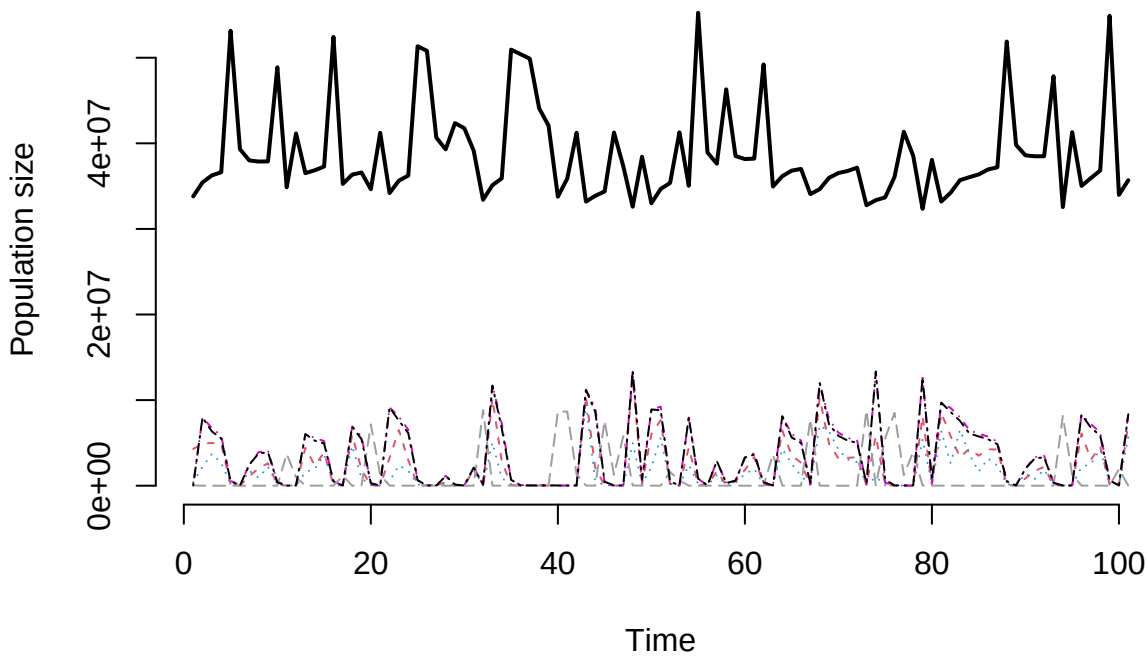

Figure 1: Population and community aggregate density in test projection.

The plot above shows the aggregate community density as the thick black line, and the constituent population sizes are the thinner lines below. The great disparity between the aggregate density and the constituent populations relates to the use of stage equivalents to make the aggregate density comparable across guilds. We can take a look at what is happening within each guild by focusing on object `structure` in our `adaptProj` object, as below where we focus on the tamarisk data.

```
par(mfrow = c(1, 2))

species <- 2

plot(lytle_alleq_alldens_proj$structure[[1]][[species]][1,], type = "l", lwd = 2,
  ylab = "No of individuals", xlab = "Time", main = "Tamarisk All Stages",
  bty = "n")
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][2,], lwd = 2, lty = 2)
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][3,], lwd = 2, lty = 3)
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][4,], lwd = 2, lty = 4)
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][5,], lwd = 2, lty = 5)
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][6,], lwd = 2, lty = 6)

plot(lytle_alleq_alldens_proj$structure[[1]][[species]][2,], type = "l", lwd = 2,
```

```
  lty = 2, ylab = "No of individuals", xlab = "Time", main = "Tamarisk No Seedlings",
  bty = "n")
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][3,], lwd = 2, lty = 3)
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][4,], lwd = 2, lty = 4)
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][5,], lwd = 2, lty = 5)
lines(lytle_alleq_alldens_proj$structure[[1]][[species]][6,], lwd = 2, lty = 6)
```

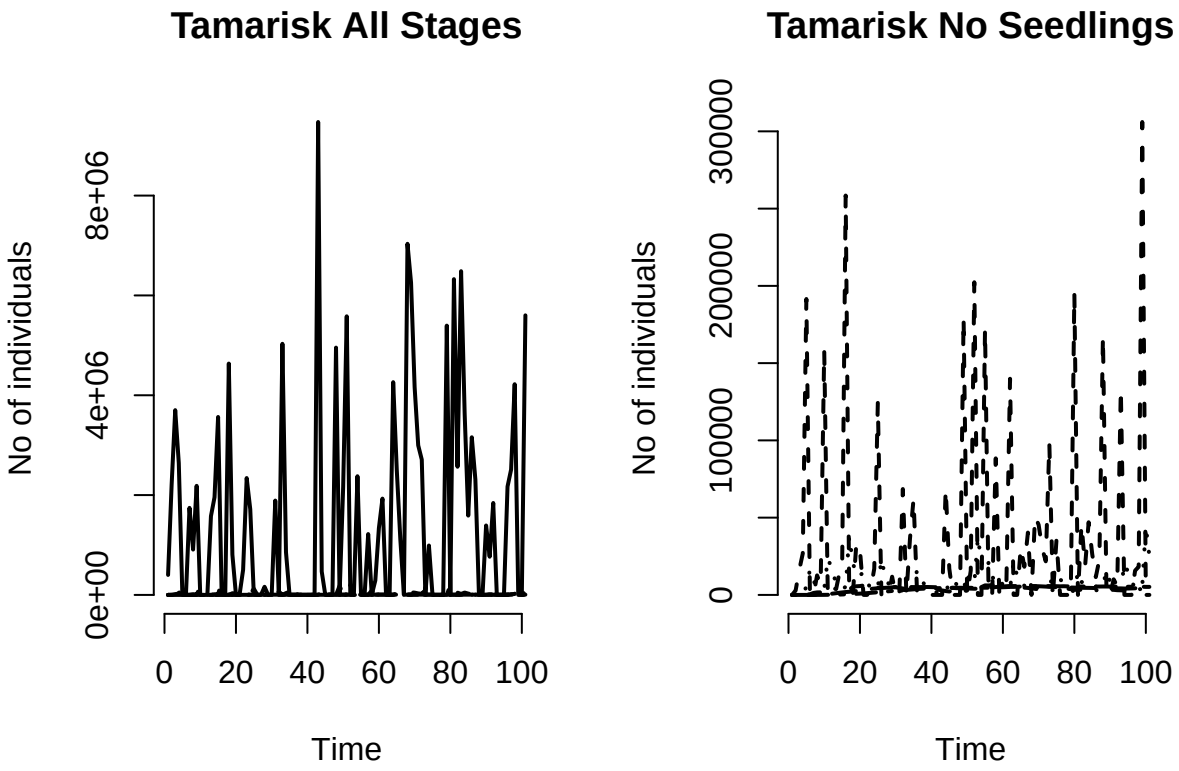

Figure 2: Tamarisk stage sizes with and without seedlings.

We made two plots above, one showing all six stages, and the other without seedlings, because the seedlings are such a large part of the population that they drown out the patterns for the other stages in the first plot.

PART 7. Recreating Drought Projection Analysis from Lytle et al. (2017)

Now we will move on to recreating some of the analyses in Lytle *et al.* (2017). Those projections were temporally stochastic. The authors assumed that each matrix had equal probability of being chosen, but they altered the hydrological parameters governing whether a flood or a drought occurred in a given year such that increasing drought frequency led to increasing numbers of matrices being assigned drought values. There matrices were reconstructed at each time step, with many of their other core values unchanged. We use a different approach in which we first identify the probability of each matrix, and then alter the matrix transition probabilities to make matrices associated with flood years or drought years more or less likely to be chosen, depending on what we wish to test. We will start by assessing the probabilities of flood years, drought years, and years without extremes, as given in the maybell data.

```
c_bigflood <- sum(bigflood)
c_drought <- sum(drought)
c_nodrought <- 83 - c_drought
c_noextremes <- 83 - (c_bigflood + c_drought)
```

```r
c_bigflood # Number of years with floods
```

```
## [1] 47
```

```r
c_drought # Number of years with droughts
```

```
## [1] 17
```

```r
c_nodrought # Number of years without drought
```

```
## [1] 66
```

```r
c_noextremes #Number of years without extremes
```

```
## [1] 19
```

```r
P_bigflood <- c_bigflood/83
P_drought <- c_drought/83
P_noextremes <- c_noextremes / 83

P_bigflood # Probability of a flood year
```

```
## [1] 0.5662651
```

```r
P_drought # Probability of a drought year
```

```
## [1] 0.2048193
```

```r
P_noextremes # Probability of a year without extremes
```

```
## [1] 0.2289157
```

Now we will test the impact of increasing drought, as the authors did though using our different approach to temporal stochasticity. We will conduct a total of 67 projections, in which we start with the base drought probability and increase the chance of drought year-associated matrices being chosen at each projection, while downweighting all others. Note that all projections here are temporally stochastic. To keep the results equivalent across computers and R sessions, make sure to run the whole block at once, particularly keeping the `set.seed()` statement as is. We also note that the following loop takes less than a minute to run on an Apple MacBook Pro with Apple M3 Max architecture.

```r
times_projected <- 200
total_reps <- 10

total_drought_variants <- total_years - c_drought + 1
stoch_vec <- rep(1/83, 83)

drought_elems_logical <- as.logical(drought)
drought_elems <- which(drought_elems_logical)
nodrought_elems <- which(!drought_elems_logical)

p_drought_stoch <- vector(mode = "numeric", length = total_drought_variants)

rip_proj_list <- vector(mode = "list", length = total_drought_variants)

set.seed(42)

for (i in c(0:(total_years - c_drought))) {
  stoch_vec_rep <- stoch_vec
  stoch_vec_rep[drought_elems] <- stoch_vec[drought_elems] + (i/total_years) / c_drought # Increases pr
```

```
  stoch_vec_rep[nodrought_elems] <- stoch_vec[nodrought_elems] - (i/total_years) / c_nodrought # Decrea

  p_drought_stoch[i + 1] <- sum(stoch_vec_rep[drought_elems])
  stoch_vec_list <- list(stoch_vec_rep, stoch_vec_rep, stoch_vec_rep, stoch_vec_rep, stoch_vec_rep)

  rip_proj_list[[i + 1]] <- project3(mpms = lytle_mpms, times = times_projected,
    nreps = total_reps, equivalence = lytle_alleq, density = lytle_alldens,
    starts = lytle_starts, stochastic = TRUE, tweights = stoch_vec_list)
}
```

We wish to reproduce Figure 2 in Lytle *et al.* (2017) as faithfully as possible, given the different assumptions we made governing how temporal stochasticity worked in our projections. To reproduce this figure, we first need to collect the population size associated with each plant population across all 67 variants (object `N_means` below). Then, we can plot the results against the vector of drought probability that we created at each run through the loop above. Note that the plot below is not yet the equivalent of Figure 2 in in Lytle *et al.* (2017), because the latter is in terms of % riparian area taken up by each population, while our plot is of the raw population sizes.

```
N_means <- apply(as.matrix(1:total_drought_variants), 1, function(X) {
  N_means_byrep <- apply(as.matrix(c(1:total_reps)), 1, function(Y) {
    apply(rip_proj_list[[X]]$N_out[[Y]][,c((times_projected - 99):(times_projected + 1))], 1, mean)
  })

  N_means_current_variant <- as.matrix(apply(N_means_byrep, 1, mean))

  return(N_means_current_variant)
})

plot(y = N_means[1,], x = p_drought_stoch, ylab = "Pop size (N)", ylim = c(0, 5e+06),
  xlab = "Drought frequency", type = "l", lty = 1, lwd = 2, bty = "n")
lines(y = N_means[2,], x = p_drought_stoch, col = "red", lty = 2, lwd = 2)
lines(y = N_means[3,], x = p_drought_stoch, col = "blue", lty = 3, lwd = 2)
lines(y = N_means[4,], x = p_drought_stoch, col = "orange", lty = 4, lwd = 2)
lines(y = N_means[5,], x = p_drought_stoch, col = "purple", lty = 5, lwd = 2)
legend("topright", lty = c(1, 2, 3, 4, 5), lwd = 2, bty = "n",
  col = c("black", "red", "blue", "orange", "purple"),
  legend = c("Cottonwood", "Tamarisk", "Willow", "Sagebrush", "Meadow"))
```

Now we will create a plot showing population size as a function of % riparian area.

```
Stage_means <- vector(mode = "list", length = total_drought_variants)

for (X in c(1:total_drought_variants)) {
  Stage_means_current_variant <- apply(as.matrix(c(1:5)), 1, function(Y) {
    Stage_means_spY_byrep <- apply(as.matrix(c(1:total_reps)), 1, function(Z) {
      apply(rip_proj_list[[X]]$structure[[Z]][[Y]][,c((times_projected - 99):(times_projected + 1))], 1
    })

    Stage_means_current_variant_spY <- as.matrix(apply(Stage_means_spY_byrep, 1, mean))

    return(Stage_means_current_variant_spY)
  })

  Stage_means[[X]] <- Stage_means_current_variant
}
```

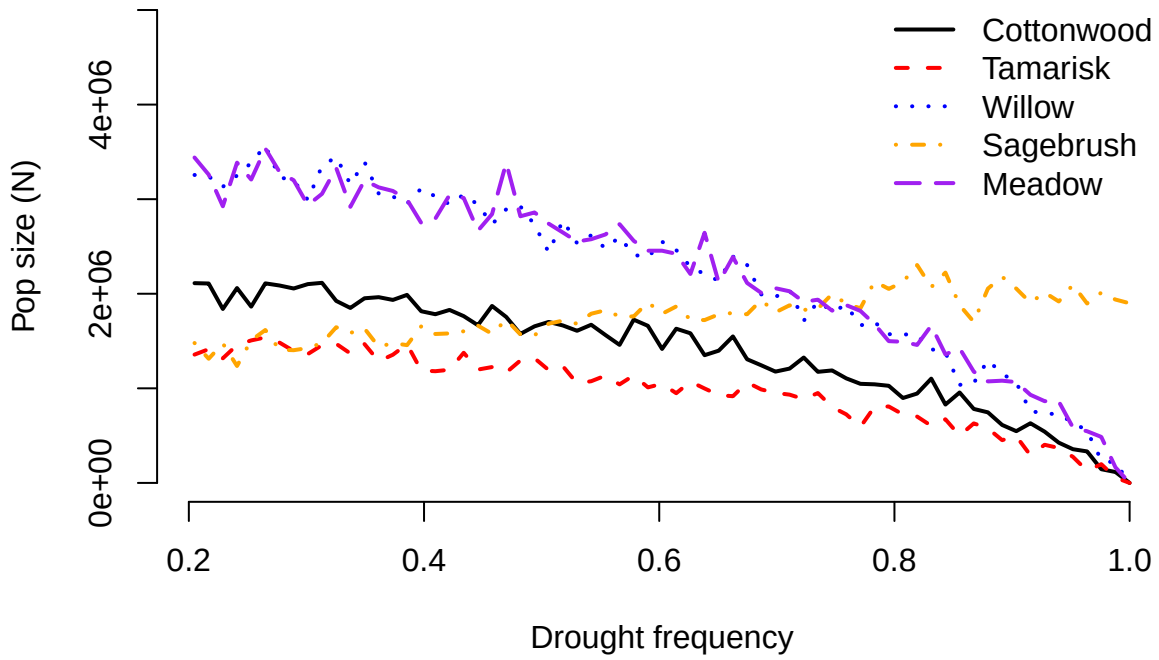

Figure 3: Guild size as a function of drought frequency.

```
Prop_means <- apply(as.matrix(c(1:total_drought_variants)), 1, function(X) {
  new_mat <- Stage_means[[X]]
  new_mat[,1] <- new_mat[,1] * as.matrix(value_vec_C_eq)
  new_mat[,2] <- new_mat[,2] * as.matrix(value_vec_T_eq)
  new_mat[,3] <- new_mat[,3] * as.matrix(value_vec_W_eq)
  new_mat[,4] <- new_mat[,4] * as.matrix(value_vec_S_eq)
  new_mat[,5] <- new_mat[,5] * as.matrix(value_vec_M_eq)

  new_vec <- apply(new_mat, 2, sum)

  final_vec <- new_vec / sum(new_vec)

  return(final_vec)
})

plot(y = Prop_means[1,], x = p_drought_stoch, ylab = "Prop bare ground",
  ylim = c(0., 1.), xlab = "Drought frequency", type = "l", lty = 1, lwd = 2,
  bty = "n")
lines(y = Prop_means[2,], x = p_drought_stoch, col = "red", lty = 2, lwd = 2)
lines(y = Prop_means[3,], x = p_drought_stoch, col = "blue", lty = 3, lwd = 2)
lines(y = Prop_means[4,], x = p_drought_stoch, col = "orange", lty = 4, lwd = 2)
lines(y = Prop_means[5,], x = p_drought_stoch, col = "purple", lty = 5, lwd = 2)
legend("topleft", lty = c(1, 2, 3, 4, 5), lwd = 2, bty = "n",
  col = c("black", "red", "blue", "orange", "purple"),
  legend = c("Cottonwood", "Tamarisk", "Willow", "Sagebrush", "Meadow"))
```

Astute study will note some difference between this figure and the published figure. Nonetheless, these differences are relatively minor and due to how temporal stochasticity was parameterized, and so relate to differences in our assumptions. The figures are qualitatively similar.

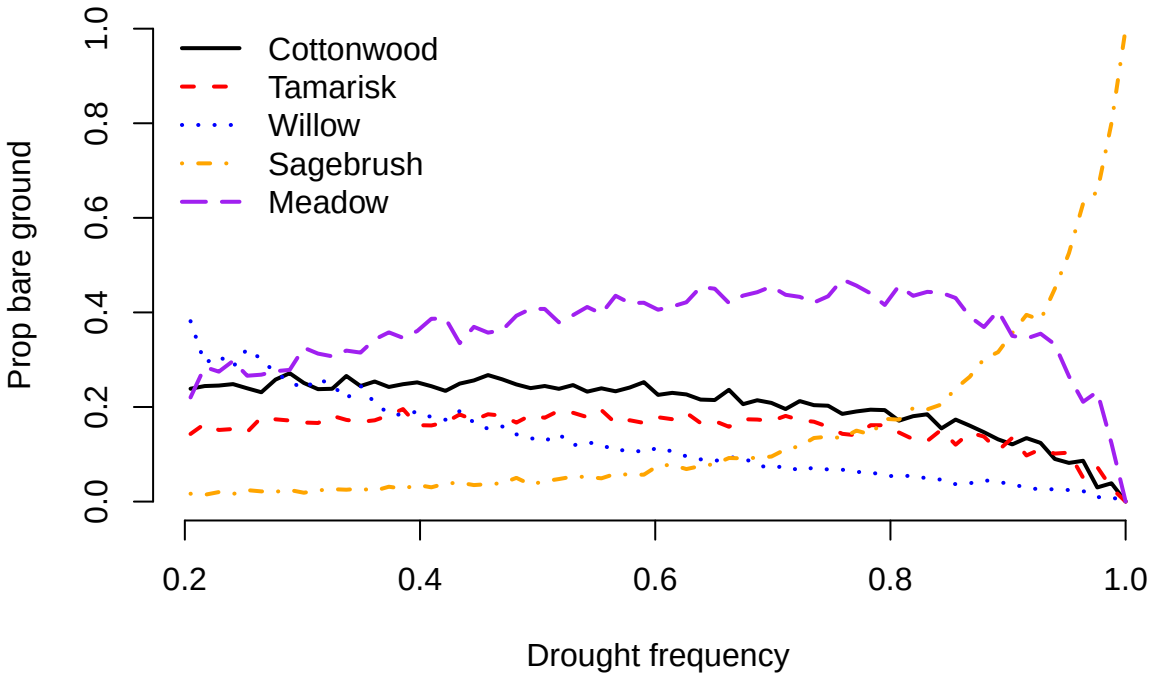

Figure 4: Proportion of bare ground occupied by each guild as a function of drought frequency

PART 8. Sensitivity Analysis

Now we will move on to sensitivity analysis. Sensitivity analysis refers to any analysis that tests how strongly key metrics of some function respond to small, additive changes in their inputs. In matrix population models, sensitivity analysis typically refers to the sensitivity of the population growth rate to changes in matrix elements, as per the formula $\frac{\partial \lambda}{\partial a_{ij}}$, where λ is the asymptotic population growth rate, and a_{ij} is the matrix element for the transition from stage j in time t to stage i in time $t+1$ (Caswell 2001). We may also have "lower-level" sensitivities refer to the strength of impacts on population growth rate to changes in the fundamental vital rates that determine matrix elements, which may be given by the formula $\frac{\partial \lambda}{\partial \phi}$, where λ is as before and ϕ is the parameter of interest.

In matrix community models, sensitivity analysis may refer to impacts on the aggregate community density, on each population density, and on other community metrics, such as species dominance. If matrix elements are determined by vital rate functions, then "lower-level" sensitivities may also be assessed here.

In Lytle *et al.* (2017), the authors conducted sensitivity analyses assessing how species dominance and guild size changed as a function of shifts both to matrix elements in the matrices of particular guilds, and to the vital rates that determined them. Redoing that analysis would be too much for this example, but we will show how to conduct both standard and lower-level sensitivity analyses using package `adapt3`.

First, we develop what might be the simplest sensitivity analysis, at least from the coding standpoint: sensitivity of population densities to changes in matrix elements. We will focus only on the survival of the oldest adults here.

In the block below, we set up a batch projection, meaning that we will set up a series of projections that will all be run together. First, we will create a series of supplements using the `lefko3` function `supplemental()`, in which we identify the matrix elements that we wish to assess sensitivity to. However, unlike in the typical use of function `supplemental()`, we will not identify the change that we wish to make to the matrix elements. Below, we identify fecundity of the oldest adult class in each case as our target. Notice that we also package the supplements in list format, in the same order as before.

```r
trial_supp_cottonwood <- supplemental(stageframe = cottonwood_lM$ahstages,
  stage3 = "adult", stage2 = "adult", type = 1, historical = FALSE)
trial_supp_tamarisk <- supplemental(stageframe = tamarisk_lM$ahstages,
  stage3 = "over30yr", stage2 = "over30yr", type = 1, historical = FALSE)
trial_supp_willow <- supplemental(stageframe = willow_lM$ahstages,
  stage3 = "over30yr", stage2 = "over30yr", type = 1, historical = FALSE)
trial_supp_sagebrush <- supplemental(stageframe = sagebrush_lM$ahstages,
  stage3 = "over30yr", stage2 = "over30yr", type = 1, historical = FALSE)
trial_supp_meadow <- supplemental(stageframe = meadow_lM$ahstages,
  stage3 = "over30yr", stage2 = "over30yr", type = 1, historical = FALSE)

post_proc_supp_list <- list(trial_supp_cottonwood, trial_supp_tamarisk,
  trial_supp_willow, trial_supp_sagebrush, trial_supp_meadow)
```

Now we will enter this supplement list into function `batch_project3()`, which runs MCM projections in batches. We also enter the argument `used_mpms = "all"`, which tells this function that we wish to apply alterations to all five MPMs in our MCM. But what alterations will we set up? The default setting is to apply additive offsets to the matrix elements, one at a time, with each offset being one of the five values (0.005, 0.010, 0.015, 0.020, 0.025). Users wishing to specify other additive offsets may do so with the `offset` argument - for example, Lytle *et al.* (2017) used 25 values in increments of 0.001 from 0.001 to 0.25. Users may instead change matrix elements completely with argument `givenrate`, or multiply elements with argument `multiplier`. The default setting of `offset = c(0.005, 0.010, 0.015, 0.020, 0.025)` was set up with sensitivity in mind.

Let's run our batch projection and look at the summary. Note that this batch projection should not take more than a few minutes (50s on a MacBook Pro with Apple M3 Max architecture).

```r
set.seed(42)

sens_reps <- 25

first_sens_run <- batch_project3(supplements = post_proc_supp_list,
  used_mpms = "all", mpms = lytle_mpms, times = times_projected,
  nreps = sens_reps, equivalence = lytle_alleq, density = lytle_alldens,
  starts = lytle_starts, stochastic = TRUE)
```

Function `batch_project3()` works by implementing a single change on a single MPM, and then projecting the full MCM with that change. It repeats this with the same change for the number of replicates specified in argument `nreps`, before moving on to the next change. The rows in our summary give both the change made (in columns, `stage3`, `stage2`, and `offset`), and the MPM in which the change was applied (in column `target_mpm`). Then it provides the final population size of that each MPM in that run (given in columns `mpm` and `final_N`). So, since we are applying five individual changes to each of our five MPMs in turn, and we are running 10 replicates and exploring the impacts of the change on each MPM's final population size, we end up with $5 \times 5 \times 25 \times 5 = 3125$ rows of output.

Let's see how our first series of changes impacted the various population sizes. To make these assessments, we will first run a summary of the output and assign it to a new variable, so that we can call it up later. We will also add a new vector that will give us the mean value of oldest adult cottonwood survival, which, given that we have performed a completely stochastic series of projections, will be simply the mean value of this survival element from all 83 matrices in each projection within the batch projection added to each offset. Then we will subset that data frame to the first five projections, which covered all five changes made to the fecundity of hydroriparian trees (i.e. cottonwood). Finally, we will subset that subset into its constituent MPMs, and then run linear regressions of the final population size of each guild against the additive offset to the fecundity of the oldest cottonwoods. We will also look at summaries of these regressions.

```r
first_sens_run_summary <- summary(first_sens_run)
```

```
## This adaptProjBatch has 25 projections covering 5 mpms, of which 5 have been targeted for alterations
## Assuming the first projection is representative, each projection is 200 time steps in length.
```

```r
cottonwood_surv_base <- apply(as.matrix(c(1:83)), 1, function(X) {
  lytle_mpms[[1]]$F[[X]][6, 6]}
)
mean_cottonwood_surv <- mean(cottonwood_surv_base)

first_sens_run_summary$cottonwood_surv <- first_sens_run_summary$offset + mean_cottonwood_surv

first_sens_run_summary_ht <- subset(first_sens_run_summary, projection < 6)
first_sens_run_summary_ht_1 <- subset(first_sens_run_summary_ht, mpm == 1)
first_sens_run_summary_ht_2 <- subset(first_sens_run_summary_ht, mpm == 2)
first_sens_run_summary_ht_3 <- subset(first_sens_run_summary_ht, mpm == 3)
first_sens_run_summary_ht_4 <- subset(first_sens_run_summary_ht, mpm == 4)
first_sens_run_summary_ht_5 <- subset(first_sens_run_summary_ht, mpm == 5)

first_sens_ht_surv_1 <- lm(final_N ~ cottonwood_surv, first_sens_run_summary_ht_1)
first_sens_ht_surv_2 <- lm(final_N ~ cottonwood_surv, first_sens_run_summary_ht_2)
first_sens_ht_surv_3 <- lm(final_N ~ cottonwood_surv, first_sens_run_summary_ht_3)
first_sens_ht_surv_4 <- lm(final_N ~ cottonwood_surv, first_sens_run_summary_ht_4)
first_sens_ht_surv_5 <- lm(final_N ~ cottonwood_surv, first_sens_run_summary_ht_5)

summary(first_sens_ht_surv_1)
```

```
##
## Call:
## lm(formula = final_N ~ cottonwood_surv, data = first_sens_run_summary_ht_1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2400190 -1865388  -597161   866210 11971423
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       2596632     566339   4.585  1.1e-05 ***
## cottonwood_surv -38146612   34151514  -1.117    0.266
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2700000 on 123 degrees of freedom
## Multiple R-squared:  0.01004,    Adjusted R-squared:  0.001993
## F-statistic: 1.248 on 1 and 123 DF,  p-value: 0.2662
```

```r
summary(first_sens_ht_surv_2)
```

```
##
## Call:
## lm(formula = final_N ~ cottonwood_surv, data = first_sens_run_summary_ht_2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1373250 -1142739  -826869   388553  9189042
```

```
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       1462490     397826   3.676 0.000352 ***
## cottonwood_surv -13975954   23989824  -0.583 0.561244
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1897000 on 123 degrees of freedom
## Multiple R-squared:  0.002752,   Adjusted R-squared:  -0.005356
## F-statistic: 0.3394 on 1 and 123 DF,  p-value: 0.5612
```

```r
summary(first_sens_ht_surv_3)
```

```
##
## Call:
## lm(formula = final_N ~ cottonwood_surv, data = first_sens_run_summary_ht_3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3557518 -2541283 -1549145   727460 13365886
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       3893556     813420   4.787 4.77e-06 ***
## cottonwood_surv -67159195   49051062  -1.369    0.173
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3878000 on 123 degrees of freedom
## Multiple R-squared:  0.01501,    Adjusted R-squared:  0.007004
## F-statistic: 1.875 on 1 and 123 DF,  p-value: 0.1734
```

```r
summary(first_sens_ht_surv_4)
```

```
##
## Call:
## lm(formula = final_N ~ cottonwood_surv, data = first_sens_run_summary_ht_4)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1318308 -1303720 -1295354  -925526 11649145
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1325653     591660   2.241   0.0268 *
## cottonwood_surv -1457092   35678466  -0.041   0.9675
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 2821000 on 123 degrees of freedom
## Multiple R-squared:  1.356e-05,  Adjusted R-squared:  -0.008116
## F-statistic: 0.001668 on 1 and 123 DF,  p-value: 0.9675
```

```r
summary(first_sens_ht_surv_5)
```

```
##
## Call:
## lm(formula = final_N ~ cottonwood_surv, data = first_sens_run_summary_ht_5)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2762169 -2499472 -1194118   945225  9579120
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       2836856     698293   4.063 8.58e-05 ***
## cottonwood_surv -11982421   42108679  -0.285    0.776
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 3329000 on 123 degrees of freedom
## Multiple R-squared:  0.0006579,  Adjusted R-squared:  -0.007467
## F-statistic: 0.08097 on 1 and 123 DF,  p-value: 0.7765
```

The sensitivity of each final population size to changes in the survival of the oldest cottonwoods is given by the slopes of these linear regressions. However, in all cases, these slopes do not differ significantly from 0. Let's take a look at a plot of the impacts of changes in survival to the final population size of cottonwoods.

```r
plot(final_N ~ offset, first_sens_run_summary_ht_1, ylab = "Final population size",
  xlab = "Additive Offset to Oldest Cottonwood Survival", bty = "n")
abline(first_sens_ht_surv_1)
```

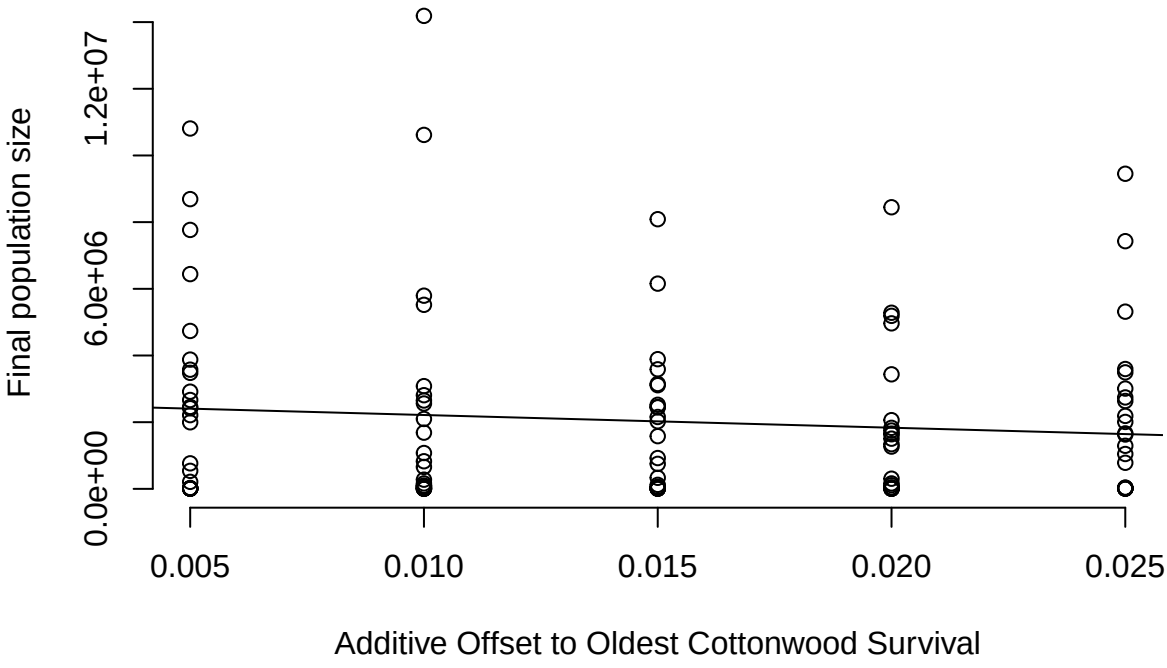

Figure 5: Final population size of cottonwood as a function of additive offset to cottonwood oldest adult survival.

We can also look at the impact of changes in survival in the oldest cottonwoods to the final population size of tamarisk, as below.

```r
plot(final_N ~ offset, first_sens_run_summary_ht_2, ylab = "Final population size",
  xlab = "Additive Offset to Oldest Cottonwood Survival", bty = "n")
abline(first_sens_ht_surv_2)
```

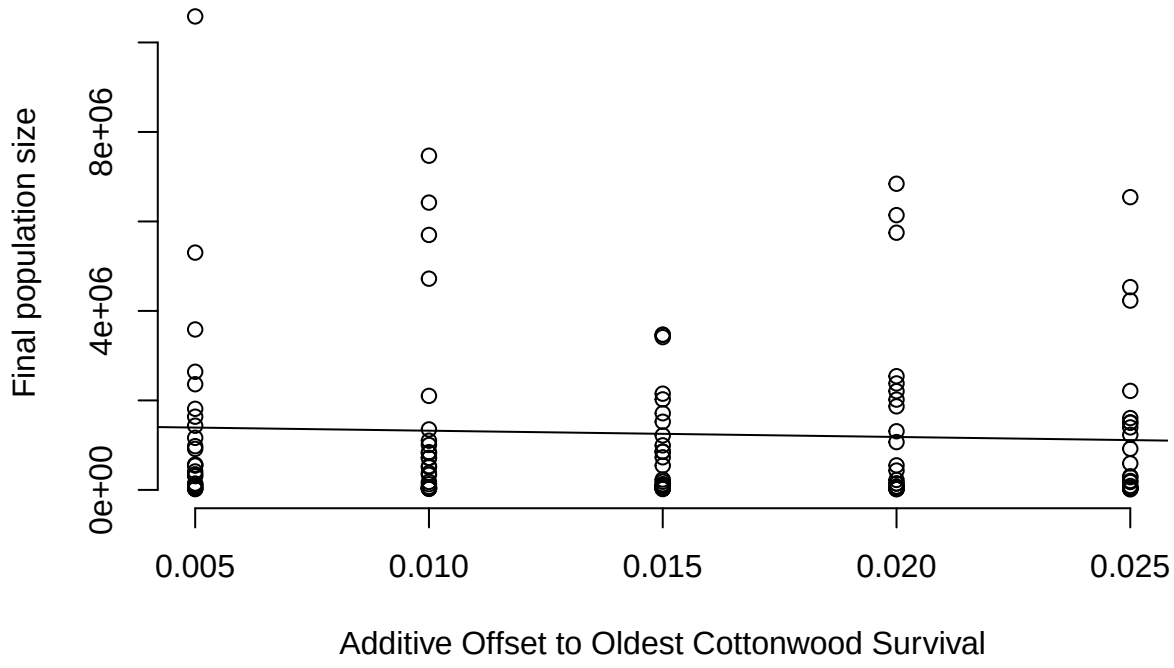

Figure 6: Final population size of tamarisk as a function of additive offset to cottonwood oldest adult survival.

Of course, the final population size in a stochastic projection, or in any projection, may be too variable to make proper inferences from. We may wish, instead, to use the mean population size for some number of time steps toward the end of the projection. Fortunately, the `summary.adaptProjBatch()` function comes to our aid in this with the `finalN_mean` and `finalN_used` arguments, where the former specifies that we would like the mean of the number of final time steps specified by the latter. The default setting for the former is `FALSE`, and the default setting for the latter is `100`. Let's alter both and re-assess the sensitivities.

```r
first_sens_run_summary_means <- summary(first_sens_run, finalN_mean = TRUE, finalN_used = 50)
```

```
## This adaptProjBatch has 25 projections covering 5 mpms, of which 5 have been targeted for alterations
## Assuming the first projection is representative, each projection is 200 time steps in length.
```

```r
first_sens_run_summary_means$cottonwood_surv <- first_sens_run_summary_means$offset + mean_cottonwood_su

first_sens_run_summary_means_ht <- subset(first_sens_run_summary_means, projection < 6)
first_sens_run_summary_means_ht_1 <- subset(first_sens_run_summary_means_ht, mpm == 1)
first_sens_run_summary_means_ht_2 <- subset(first_sens_run_summary_means_ht, mpm == 2)
first_sens_run_summary_means_ht_3 <- subset(first_sens_run_summary_means_ht, mpm == 3)
first_sens_run_summary_means_ht_4 <- subset(first_sens_run_summary_means_ht, mpm == 4)
first_sens_run_summary_means_ht_5 <- subset(first_sens_run_summary_means_ht, mpm == 5)

first_sens_ht_means_surv_1 <- lm(final_N_mean ~ cottonwood_surv, first_sens_run_summary_means_ht_1)
first_sens_ht_means_surv_2 <- lm(final_N_mean ~ cottonwood_surv, first_sens_run_summary_means_ht_2)
first_sens_ht_means_surv_3 <- lm(final_N_mean ~ cottonwood_surv, first_sens_run_summary_means_ht_3)
first_sens_ht_means_surv_4 <- lm(final_N_mean ~ cottonwood_surv, first_sens_run_summary_means_ht_4)
first_sens_ht_means_surv_5 <- lm(final_N_mean ~ cottonwood_surv, first_sens_run_summary_means_ht_5)
```

```r
summary(first_sens_ht_means_surv_1)
```

```
##
## Call:
## lm(formula = final_N_mean ~ cottonwood_surv, data = first_sens_run_summary_means_ht_1)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -872732 -297461  -47328  338745 1365225
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       2032010      97340  20.875   <2e-16 ***
## cottonwood_surv -10657638    5869838  -1.816   0.0719 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 464100 on 123 degrees of freedom
## Multiple R-squared:  0.0261, Adjusted R-squared:  0.01818
## F-statistic: 3.297 on 1 and 123 DF,  p-value: 0.07186
```

```r
summary(first_sens_ht_means_surv_2)
```

```
##
## Call:
## lm(formula = final_N_mean ~ cottonwood_surv, data = first_sens_run_summary_means_ht_2)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -825197 -250931  -36120  210575 1017294
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       1513196      77315   19.57   <2e-16 ***
## cottonwood_surv -10863458    4662260   -2.33   0.0214 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 368600 on 123 degrees of freedom
## Multiple R-squared:  0.04227,    Adjusted R-squared:  0.03449
## F-statistic: 5.429 on 1 and 123 DF,  p-value: 0.02143
```

```r
summary(first_sens_ht_means_surv_3)
```

```
##
## Call:
## lm(formula = final_N_mean ~ cottonwood_surv, data = first_sens_run_summary_means_ht_3)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1176314  -339633    -1588   306701  1423963
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       3402024     102025   33.34  < 2e-16 ***
```

```
## cottonwood_surv -18762222    6152343   -3.05  0.00281 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 486400 on 123 degrees of freedom
## Multiple R-squared:  0.0703, Adjusted R-squared:  0.06274
## F-statistic:   9.3 on 1 and 123 DF,  p-value: 0.002807
```

```r
summary(first_sens_ht_means_surv_4)
```

```
##
## Call:
## lm(formula = final_N_mean ~ cottonwood_surv, data = first_sens_run_summary_means_ht_4)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -887157 -289769  -21680  260566 1014477
##
## Coefficients:
##                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)      1388466      87464  15.875   <2e-16 ***
## cottonwood_surv -4689483    5274279  -0.889    0.376
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 417000 on 123 degrees of freedom
## Multiple R-squared:  0.006386,   Adjusted R-squared:  -0.001692
## F-statistic: 0.7905 on 1 and 123 DF,  p-value: 0.3757
```

```r
summary(first_sens_ht_means_surv_5)
```

```
##
## Call:
## lm(formula = final_N_mean ~ cottonwood_surv, data = first_sens_run_summary_means_ht_5)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1661529  -545735   -53448   453152  2408684
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       3375332     158789  21.257   <2e-16 ***
## cottonwood_surv -24564296    9575333  -2.565   0.0115 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 757000 on 123 degrees of freedom
## Multiple R-squared:  0.05079,    Adjusted R-squared:  0.04307
## F-statistic: 6.581 on 1 and 123 DF,  p-value: 0.01151
```

It appears that the survival of the oldest cottonwoods does indeed have an influence on this community, particularly through impacts on tamarisks, willows, and meadow plants. Let's take a look at a plot for tamarisks.

```r
plot(final_N ~ offset, first_sens_run_summary_means_ht_2, ylab = "Mean final population size",
  xlab = "Additive Offset to Cottonwood Survival", bty = "n")
```

```
abline(first_sens_ht_means_surv_2)
```

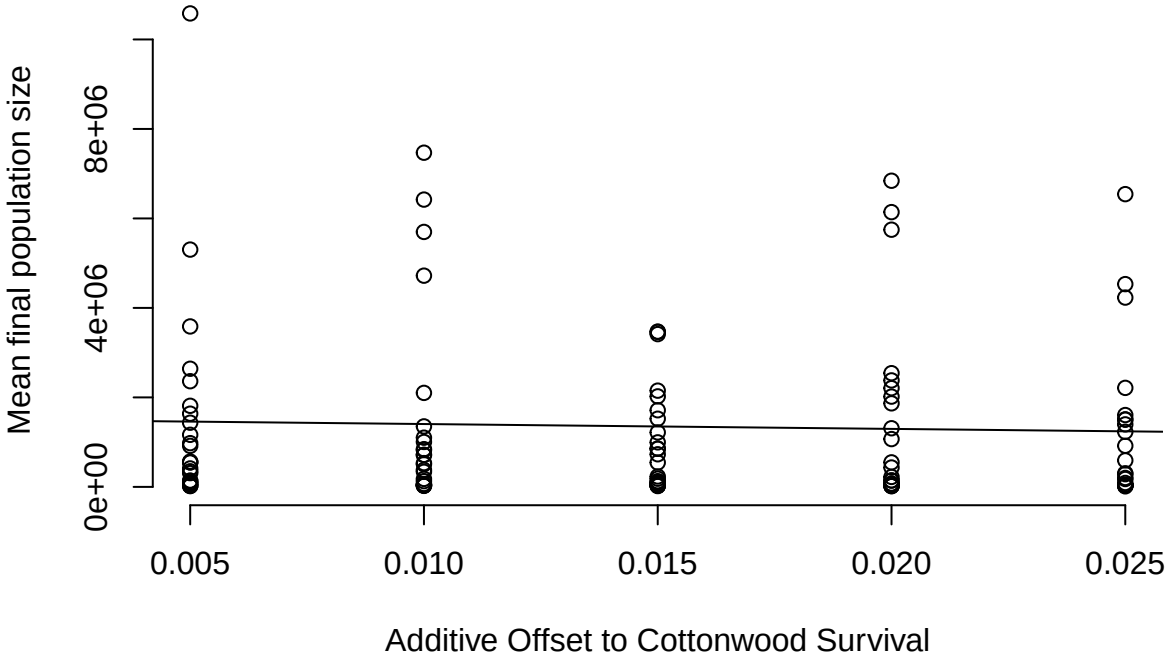

Figure 7: Mean final population size of tamarisk as a function of additive offset to cottonwood oldest adult survival.

For more complex sensitivities, users will need to go into the list output by function `batch_project3()`.

In addition to standard matrix element sensitivities, users may wish to conduct "lower-level" sensitivities. The MCM from Lytle *et al.* (2017) is a perfect example to learn from, because the matrices packaged into the MCM are all developed from a series of functions in which matrix elements are actually functions of a series of demographic and environmental variables. Indeed, the authors conducted such a sensitivity analysis by focusing on the impacts of changes to what they termed the "flood mortality in a flood year" (S), the "drought mortality in a drought year", and the "baseline transition probability". We will not conduct all of these analyses, but will instead focus just on determining the sensitivities of mean final population size of our plant guilds to changes in the drought mortality associated with cottonwoods.

In the block below, we create five new sets of MCMs, using the `matrix_creator()` function that we developed earlier. In each case, we alter drought mortality associated with cottonwoods by a small, additive offset.

```
riparian_mats_ht_drought_1 <- matrix_creator(flowdata, total_years,
  DC6_offset = 0.005)
riparian_mats_ht_drought_2 <- matrix_creator(flowdata, total_years,
  DC6_offset = 0.010)
riparian_mats_ht_drought_3 <- matrix_creator(flowdata, total_years,
  DC6_offset = 0.015)
riparian_mats_ht_drought_4 <- matrix_creator(flowdata, total_years,
  DC6_offset = 0.020)
riparian_mats_ht_drought_5 <- matrix_creator(flowdata, total_years,
  DC6_offset = 0.025)

cottonwood_lM_1 <- create_lM(riparian_mats_ht_drought_1$cottonwood, tree_frame,
  yearorder = c(1:83))
tamarisk_lM_1 <- create_lM(riparian_mats_ht_drought_1$tamarisk, mead_frame,
```

```r
  yearorder = c(1:83))
sagebrush_lM_1 <- create_lM(riparian_mats_ht_drought_1$sagebrush, mead_frame,
  yearorder = c(1:83))
meadow_lM_1 <- create_lM(riparian_mats_ht_drought_1$meadow, mead_frame,
  yearorder = c(1:83))
willow_lM_1 <- create_lM(riparian_mats_ht_drought_1$willow, will_frame,
  yearorder = c(1:83))

cottonwood_lM_2 <- create_lM(riparian_mats_ht_drought_2$cottonwood, tree_frame,
  yearorder = c(1:83))
tamarisk_lM_2 <- create_lM(riparian_mats_ht_drought_2$tamarisk, mead_frame,
  yearorder = c(1:83))
sagebrush_lM_2 <- create_lM(riparian_mats_ht_drought_2$sagebrush, mead_frame,
  yearorder = c(1:83))
meadow_lM_2 <- create_lM(riparian_mats_ht_drought_2$meadow, mead_frame,
  yearorder = c(1:83))
willow_lM_2 <- create_lM(riparian_mats_ht_drought_2$willow, will_frame,
  yearorder = c(1:83))

cottonwood_lM_3 <- create_lM(riparian_mats_ht_drought_3$cottonwood, tree_frame,
  yearorder = c(1:83))
tamarisk_lM_3 <- create_lM(riparian_mats_ht_drought_3$tamarisk, mead_frame,
  yearorder = c(1:83))
sagebrush_lM_3 <- create_lM(riparian_mats_ht_drought_3$sagebrush, mead_frame,
  yearorder = c(1:83))
meadow_lM_3 <- create_lM(riparian_mats_ht_drought_3$meadow, mead_frame,
  yearorder = c(1:83))
willow_lM_3 <- create_lM(riparian_mats_ht_drought_3$willow, will_frame,
  yearorder = c(1:83))

cottonwood_lM_4 <- create_lM(riparian_mats_ht_drought_4$cottonwood, tree_frame,
  yearorder = c(1:83))
tamarisk_lM_4 <- create_lM(riparian_mats_ht_drought_4$tamarisk, mead_frame,
  yearorder = c(1:83))
sagebrush_lM_4 <- create_lM(riparian_mats_ht_drought_4$sagebrush, mead_frame,
  yearorder = c(1:83))
meadow_lM_4 <- create_lM(riparian_mats_ht_drought_4$meadow, mead_frame,
  yearorder = c(1:83))
willow_lM_4 <- create_lM(riparian_mats_ht_drought_4$willow, will_frame,
  yearorder = c(1:83))

cottonwood_lM_5 <- create_lM(riparian_mats_ht_drought_5$cottonwood, tree_frame,
  yearorder = c(1:83))
tamarisk_lM_5 <- create_lM(riparian_mats_ht_drought_5$tamarisk, mead_frame,
  yearorder = c(1:83))
sagebrush_lM_5 <- create_lM(riparian_mats_ht_drought_5$sagebrush, mead_frame,
  yearorder = c(1:83))
meadow_lM_5 <- create_lM(riparian_mats_ht_drought_5$meadow, mead_frame,
  yearorder = c(1:83))
willow_lM_5 <- create_lM(riparian_mats_ht_drought_5$willow, will_frame,
  yearorder = c(1:83))

lytle_mpms_1 <- list(cottonwood_lM_1, tamarisk_lM_1, willow_lM_1,
```

```
  sagebrush_lM_1, meadow_lM_1)
lytle_mpms_2 <- list(cottonwood_lM_2, tamarisk_lM_2, willow_lM_2,
  sagebrush_lM_2, meadow_lM_2)
lytle_mpms_3 <- list(cottonwood_lM_3, tamarisk_lM_3, willow_lM_3,
  sagebrush_lM_3, meadow_lM_3)
lytle_mpms_4 <- list(cottonwood_lM_4, tamarisk_lM_4, willow_lM_4,
  sagebrush_lM_4, meadow_lM_4)
lytle_mpms_5 <- list(cottonwood_lM_5, tamarisk_lM_5, willow_lM_5,
  sagebrush_lM_5, meadow_lM_5)
```

Now we will run five separate projections, each with a series of replicates.

```
Dsens_proj_1 <- project3(mpms = lytle_mpms_1, times = times_projected,
    nreps = sens_reps, equivalence = lytle_alleq, density = lytle_alldens,
    starts = lytle_starts, stochastic = TRUE)
Dsens_proj_2 <- project3(mpms = lytle_mpms_2, times = times_projected,
    nreps = sens_reps, equivalence = lytle_alleq, density = lytle_alldens,
    starts = lytle_starts, stochastic = TRUE)
Dsens_proj_3 <- project3(mpms = lytle_mpms_3, times = times_projected,
    nreps = sens_reps, equivalence = lytle_alleq, density = lytle_alldens,
    starts = lytle_starts, stochastic = TRUE)
Dsens_proj_4 <- project3(mpms = lytle_mpms_4, times = times_projected,
    nreps = sens_reps, equivalence = lytle_alleq, density = lytle_alldens,
    starts = lytle_starts, stochastic = TRUE)
Dsens_proj_5 <- project3(mpms = lytle_mpms_5, times = times_projected,
    nreps = sens_reps, equivalence = lytle_alleq, density = lytle_alldens,
    starts = lytle_starts, stochastic = TRUE)
```

The function `summary.adaptProj()` can help us make sense of the output. As before, we can extract both the final population size of each guild, as well as the mean final population size. Let's do that, as below.

```
summary(Dsens_proj_1, finalN_mean = TRUE, finalN_used = 50)
```

```
##
## The input adaptProj object covers 5 populations, 201 projected steps and 25 projected replicates.
## The number of replicates with population size above the threshold size of 1 is as in
## the following matrix, with populations given by row and milepost times given by column:

## $milepost_sums
##       1 50 100 150 201
## [1,] 25 25  25  25  25
## [2,] 25 25  25  25  25
## [3,] 25 25  25  25  25
## [4,] 25 25  25  25  25
## [5,] 25 25  25  25  25
##
## $extinction_times
## [1] NA
##
## $final_N
##       1       2       3       4       5
## 1985623 1514608 3453526 1446789 3399079
```

```
summary(Dsens_proj_2, finalN_mean = TRUE, finalN_used = 50)
```

```
##
```

```
## The input adaptProj object covers 5 populations, 201 projected steps and 25 projected replicates.
## The number of replicates with population size above the threshold size of 1 is as in
## the following matrix, with populations given by row and milepost times given by column:

## $milepost_sums
##       1 50 100 150 201
## [1,] 25 25  25  25  25
## [2,] 25 25  25  25  25
## [3,] 25 25  25  25  25
## [4,] 25 25  25  25  25
## [5,] 25 25  25  25  25
##
## $extinction_times
## [1] NA
##
## $final_N
##       1       2       3       4       5
## 2365479 1636249 3390001 1309151 3382179
```

```r
summary(Dsens_proj_3, finalN_mean = TRUE, finalN_used = 50)
```

```
##
## The input adaptProj object covers 5 populations, 201 projected steps and 25 projected replicates.
## The number of replicates with population size above the threshold size of 1 is as in
## the following matrix, with populations given by row and milepost times given by column:

## $milepost_sums
##       1 50 100 150 201
## [1,] 25 25  25  25  25
## [2,] 25 25  25  25  25
## [3,] 25 25  25  25  25
## [4,] 25 25  25  25  25
## [5,] 25 25  25  25  25
##
## $extinction_times
## [1] NA
##
## $final_N
##       1       2       3       4       5
## 2230596 1412279 3251614 1357640 3317340
```

```r
summary(Dsens_proj_4, finalN_mean = TRUE, finalN_used = 50)
```

```
##
## The input adaptProj object covers 5 populations, 201 projected steps and 25 projected replicates.
## The number of replicates with population size above the threshold size of 1 is as in
## the following matrix, with populations given by row and milepost times given by column:

## $milepost_sums
##       1 50 100 150 201
## [1,] 25 25  25  25  25
## [2,] 25 25  25  25  25
## [3,] 25 25  25  25  25
## [4,] 25 25  25  25  25
## [5,] 25 25  25  25  25
##
## $extinction_times
```

```
## [1] NA
##
## $final_N
##       1       2       3       4       5
## 2007127 1503812 3309158 1376044 3167565
```

```r
summary(Dsens_proj_5, finalN_mean = TRUE, finalN_used = 50)
```

```
##
## The input adaptProj object covers 5 populations, 201 projected steps and 25 projected replicates.
## The number of replicates with population size above the threshold size of 1 is as in
## the following matrix, with populations given by row and milepost times given by column:

## $milepost_sums
##       1 50 100 150 201
## [1,] 25 25  25  25  25
## [2,] 25 25  25  25  25
## [3,] 25 25  25  25  25
## [4,] 25 25  25  25  25
## [5,] 25 25  25  25  25
##
## $extinction_times
## [1] NA
##
## $final_N
##       1       2       3       4       5
## 1962358 1384628 3156821 1320569 3109966
```

Although the summaries are certainly interesting, they show the mean final population size not simply as the mean of the final 50 time steps here, but also across all of the replicates. We would naturally like to extract that data in order to run linear regressions of the mean final population size against the offset. Below, we extract those data, and also develop the necessary data frames for further analysis.

```r
Dsens_proj_1_finalNmeans <- as.vector(apply(as.matrix(c(1:sens_reps)), 1, function(X) {
  apply(Dsens_proj_1$N_out[[X]][,c(151:201)], 1, mean)
}))
Dsens_proj_2_finalNmeans <- as.vector(apply(as.matrix(c(1:sens_reps)), 1, function(X) {
  apply(Dsens_proj_2$N_out[[X]][,c(151:201)], 1, mean)
}))
Dsens_proj_3_finalNmeans <- as.vector(apply(as.matrix(c(1:sens_reps)), 1, function(X) {
  apply(Dsens_proj_3$N_out[[X]][,c(151:201)], 1, mean)
}))
Dsens_proj_4_finalNmeans <- as.vector(apply(as.matrix(c(1:sens_reps)), 1, function(X) {
  apply(Dsens_proj_4$N_out[[X]][,c(151:201)], 1, mean)
}))
Dsens_proj_5_finalNmeans <- as.vector(apply(as.matrix(c(1:sens_reps)), 1, function(X) {
  apply(Dsens_proj_5$N_out[[X]][,c(151:201)], 1, mean)
}))

Dsens_proj_mpm_ids <- rep(seq(from = 1, to = 5), sens_reps)

Dsens_proj_1_df <- data.frame(proj = 1, offset = 0.005, mpm = Dsens_proj_mpm_ids,
  finalN_mean = Dsens_proj_1_finalNmeans)
Dsens_proj_2_df <- data.frame(proj = 1, offset = 0.010, mpm = Dsens_proj_mpm_ids,
  finalN_mean = Dsens_proj_2_finalNmeans)
Dsens_proj_3_df <- data.frame(proj = 1, offset = 0.015, mpm = Dsens_proj_mpm_ids,
```

```r
  finalN_mean = Dsens_proj_3_finalNmeans)
Dsens_proj_4_df <- data.frame(proj = 1, offset = 0.020, mpm = Dsens_proj_mpm_ids,
  finalN_mean = Dsens_proj_4_finalNmeans)
Dsens_proj_5_df <- data.frame(proj = 1, offset = 0.025, mpm = Dsens_proj_mpm_ids,
  finalN_mean = Dsens_proj_5_finalNmeans)

Dsens_proj_df <- rbind.data.frame(Dsens_proj_1_df, Dsens_proj_2_df,
  Dsens_proj_3_df, Dsens_proj_4_df, Dsens_proj_5_df)
Dsens_proj_df$drought_mortality <- Dsens_proj_df$offset + 0.05 # This term comes from term DC6 in the m
```

We now have a data frame assessing the result. Let's now conduct the linear regressions of the mean final population size against the offset, by guild.

```r
Dsens_proj_df_cottonwood <- subset(Dsens_proj_df, mpm == 1)
Dsens_proj_df_tamarisk <- subset(Dsens_proj_df, mpm == 2)
Dsens_proj_df_willow <- subset(Dsens_proj_df, mpm == 3)
Dsens_proj_df_sagebrush <- subset(Dsens_proj_df, mpm == 4)
Dsens_proj_df_meadow <- subset(Dsens_proj_df, mpm == 5)

Dsens_prof_df_cottonwood_sens <- lm(finalN_mean ~ drought_mortality, Dsens_proj_df_cottonwood)
Dsens_prof_df_tamarisk_sens <- lm(finalN_mean ~ drought_mortality, Dsens_proj_df_tamarisk)
Dsens_prof_df_willow_sens <- lm(finalN_mean ~ drought_mortality, Dsens_proj_df_willow)
Dsens_prof_df_sagebrush_sens <- lm(finalN_mean ~ drought_mortality, Dsens_proj_df_sagebrush)
Dsens_prof_df_meadow_sens <- lm(finalN_mean ~ drought_mortality, Dsens_proj_df_meadow)

summary(Dsens_prof_df_cottonwood_sens)
```

```
##
## Call:
## lm(formula = finalN_mean ~ drought_mortality, data = Dsens_proj_df_cottonwood)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -996763 -350759 -106853  320647 1545291
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)        2635465     425104    6.20 7.85e-09 ***
## drought_mortality -8127546    6501708   -1.25    0.214
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 514000 on 123 degrees of freedom
## Multiple R-squared:  0.01255,    Adjusted R-squared:  0.004517
## F-statistic: 1.563 on 1 and 123 DF,  p-value: 0.2136
```

```r
summary(Dsens_prof_df_tamarisk_sens)
```

```
##
## Call:
## lm(formula = finalN_mean ~ drought_mortality, data = Dsens_proj_df_tamarisk)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -728625 -233678  -23658  203745  962401
```

```
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)        1997000     278821   7.162 6.36e-11 ***
## drought_mortality -7899578    4264396  -1.852   0.0664 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 337100 on 123 degrees of freedom
## Multiple R-squared:  0.02714,    Adjusted R-squared:  0.01923
## F-statistic: 3.432 on 1 and 123 DF,  p-value: 0.06636
```

```r
summary(Dsens_prof_df_willow_sens)
```

```
##
## Call:
## lm(formula = finalN_mean ~ drought_mortality, data = Dsens_proj_df_willow)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1035204  -336417     6274   287179  1526241
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)         4270175     392164  10.889   <2e-16 ***
## drought_mortality -14828827    5997902  -2.472   0.0148 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 474200 on 123 degrees of freedom
## Multiple R-squared:  0.04734,    Adjusted R-squared:  0.0396
## F-statistic: 6.112 on 1 and 123 DF,  p-value: 0.01479
```

```r
summary(Dsens_prof_df_sagebrush_sens)
```

```
##
## Call:
## lm(formula = finalN_mean ~ drought_mortality, data = Dsens_proj_df_sagebrush)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -823039 -322602  -34932  249780 1227273
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)        1633469     357478   4.569 1.17e-05 ***
## drought_mortality -4165647    5467409  -0.762    0.448
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 432200 on 123 degrees of freedom
## Multiple R-squared:  0.004697,   Adjusted R-squared:  -0.003395
## F-statistic: 0.5805 on 1 and 123 DF,  p-value: 0.4476
```

```r
summary(Dsens_prof_df_meadow_sens)
```

```
##
## Call:
## lm(formula = finalN_mean ~ drought_mortality, data = Dsens_proj_df_meadow)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1538106  -507792     1410   430628  1500042
##
## Coefficients:
##                    Estimate Std. Error t value Pr(>|t|)
## (Intercept)         4421092     565713   7.815 2.09e-12 ***
## drought_mortality -17470893    8652228  -2.019   0.0456 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 684000 on 123 degrees of freedom
## Multiple R-squared:  0.03209,    Adjusted R-squared:  0.02422
## F-statistic: 4.077 on 1 and 123 DF,  p-value: 0.04564
```

The results suggest significant influences of changes to the drought mortality in cottonwoods on the mean final population sizes of willows (hydroriparian shrubs) and meadow plants. Let's take plot these results, first for willows.

```r
plot(finalN_mean ~ drought_mortality, Dsens_proj_df_willow, ylab = "Mean final population size",
  xlab = "Cottonwood Drought Mortality", bty = "n")
abline(Dsens_prof_df_willow_sens)
```

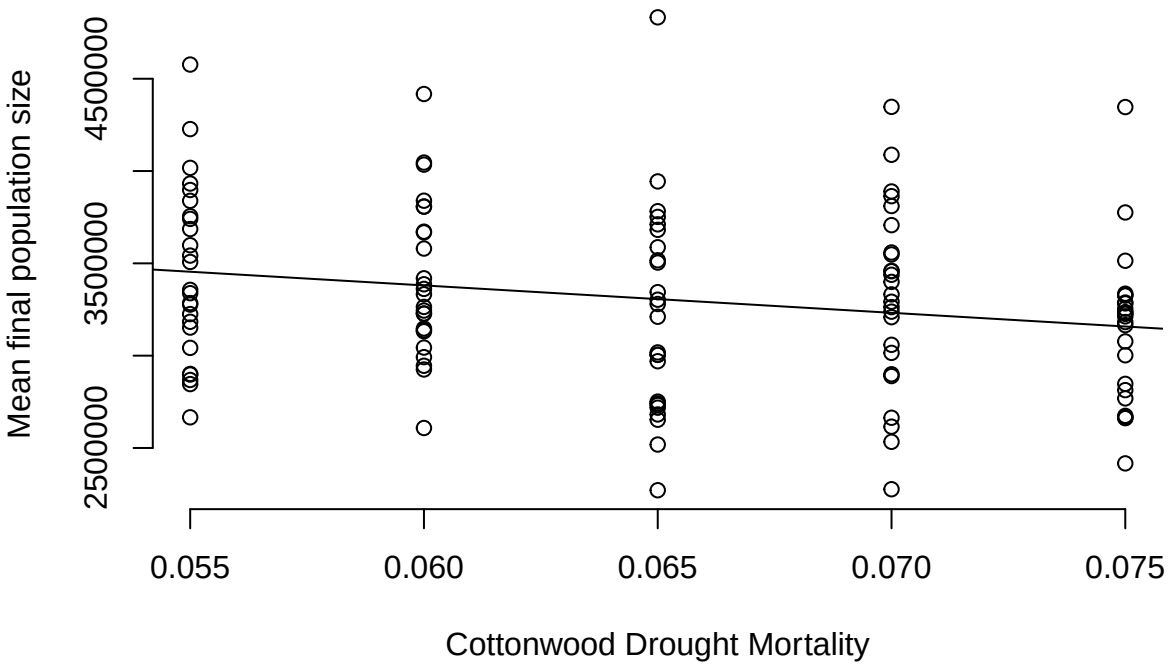

Figure 8: Mean final population size of willow as a function of cottonwood drought mortality.

We see a significant negative influence in the plot, meaning that as cottonwood drought mortality increases, the final population size of willows decreases. Let's now also take a look at the impact on meadow plants.

```r
plot(finalN_mean ~ drought_mortality, Dsens_proj_df_meadow, ylab = "Mean final population size",
  xlab = "Cottonwood Drought Mortality", bty = "n")
abline(Dsens_prof_df_meadow_sens)
```

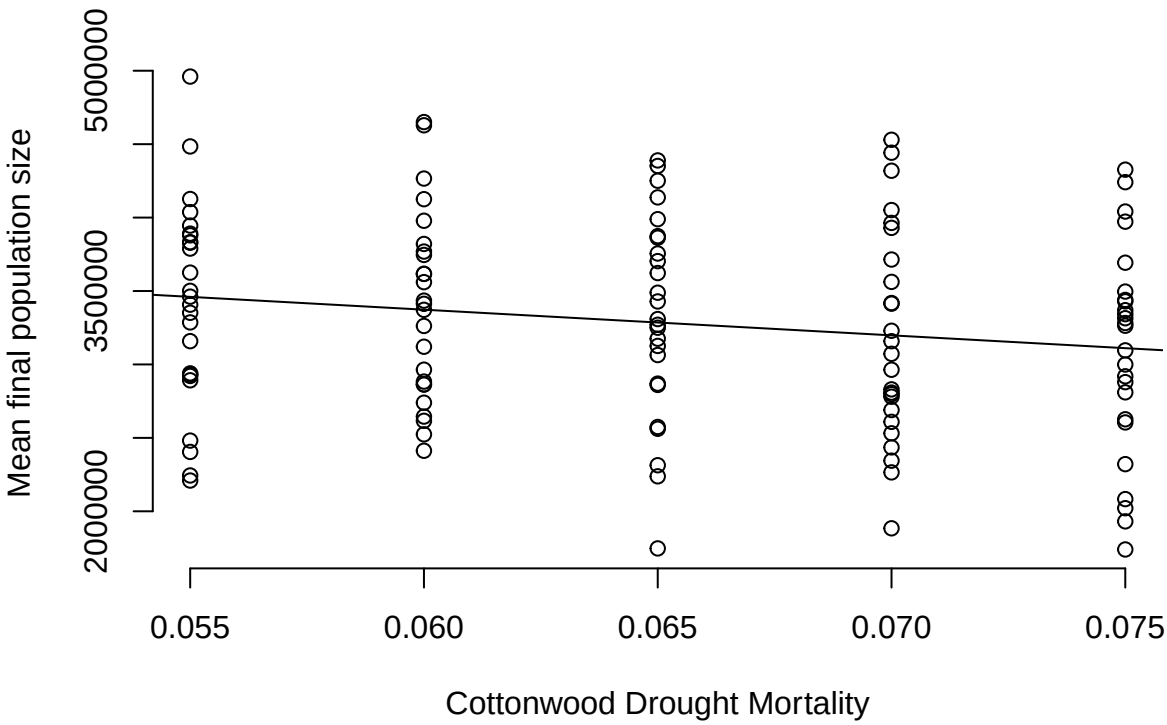

Figure 9: Mean final population size of meadow plants as a function of cottonwood drought mortality.

Once again, we see a negative impact. If we take the slopes of these regressions to be equal to the sensitivity of the mean final population size to cottonwood drought mortality, then these sensitivity terms are equal to the following.

```r
Dsens_prof_df_willow_sens$coefficients
```

```
##       (Intercept) drought_mortality
##           4270175         -14828827
```

The sensitivity protocols, batch projection functions, and other aspects of package `adapt3` should be of interest to population and community ecologists interested in working further with MPMs. We will continue to add further functionality in this package with time.

Literature Cited

Caswell, H. (2001). *Matrix population models: Construction, analysis, and interpretation*. Second edition. Sinauer Associates, Inc., Sunderland, Massachusetts, USA.

Lytle, D.A., Merritt, D.M., Tonkin, J.D., Olden, J.D. & Reynolds, L.V. (2017). Linking river flow regimes to riparian plant guilds: A community-wide modeling approach. *Ecological Applications*, 27, 1338–1350.

Mahoney, J.M. & Rood, S.B. (1998). Streamflow requirements for cottonwood seedling recruitment—an integrative model. *Wetlands*, 18, 634–645.